

**AWS VPC ARCHITECTURE WITH TWO SUBNETS,  
ROUTE TABLES, AND NAT GATEWAY**

*A Mini Project Thesis Submitted  
in the partial fulfillment of the requirements for the  
award of the degree of*

**BACHELOR OF TECHNOLOGY**

*In*

**COMPUTER SCIENCE AND ENGINEERING**



**By**

|                             |                      |
|-----------------------------|----------------------|
| <b>Pittala Haritha</b>      | <b>-(22281A05E7)</b> |
| <b>Ponnoju Ashwitha</b>     | <b>-(22281A05I4)</b> |
| <b>Yellenki Rushitha</b>    | <b>-(22281A05I7)</b> |
| <b>Gundukonda Pravalika</b> | <b>-(22281A05D3)</b> |

**PROJECT GUIDE**  
**Mrs. Fathima Anjum**  
**(Assistant Professor, CSE)**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**  
**KAMALA INSTITUTE OF TECHNOLOGY & SCIENCE**  
Approved by AICTE, New Delhi, Affiliated to JNTU, Hyderabad, T.S.,  
(UGC AUTONOMOUS)  
Accredited by NBA (CSE, ECE &EEE) and NAAC with A++  
Singapur, Huzurabad , Karimnagar, Telangana- 505468  
2024– 2025



## KAMALA INSTITUTE OF TECHNOLOGY & SCIENCE, SINGAPUR

Sponsored by KAMALA EDUCATION SOCIETY, Approved by AICTE, New Delhi and Affiliated to JNTU-Hyderabad, T.S.

(UGC AUTONOMOUS)

Accredited by NBA (CSE, ECE &EEE) and NAAC with A++

HUZURABAD, KARIMNAGAR, TELANGANA, INDIA- 505468

## Department of Computer Science & Engineering

### CERTIFICATE

This is to certify that **PITTALA HARITHA (22281A05E7), PONNOJU ASHWITHA (22281A05I4), YELLEKI RUSHITHA (22281A05I7), GUNDUKONDA PRAVALIKA (22281A05D3)** of the III B. Tech (CSE) have satisfactorily completed the dissertation work for Industrial Oriented Mini Project Work entitled **“AWS VPC ARCHITECTURE WITH TWO SUBNETS, ROUTE TABLES, AND NAT GATEWAY”** towards the partial fulfillment of B. Tech degree in the academic year 2024-2025.

**Project Guide**

**Head of the Department**

**Principal**

**External Examiner**

# INDEX

| TOPIC NO.  | TITLE                                      | PAGE NO. |
|------------|--------------------------------------------|----------|
|            | ACKNOWLEDGEMENT                            | I        |
|            | ABSTRACT                                   | II       |
|            | LIST OF FIGURES                            | III&IV   |
| CHAPTER 1: | INTRODUCTION                               | 1        |
| 1.1        | About Project                              | 1        |
| 1.2        | Existing system with drawbacks             | 1        |
| 1.3        | Proposed System with features              | 2        |
| CHAPTER 2: | LITERATURE SURVEY                          | 3        |
| CHAPTER 3: | ANALYSIS                                   | 5        |
| 3.1        | Hardware and Software requirements         | 5        |
| 3.2        | Functional and Non-functional requirements | 5        |
| 3.3        | Module Description                         | 6        |
| CHAPTER 4: | DESIGN                                     | 8        |
| 4.1        | Block diagram                              | 8        |
| 4.2        | DFD                                        | 9        |
| 4.3        | UML diagrams                               | 9        |
| 4.3.1      | Class diagram                              | 9        |
| 4.3.2      | Use case diagram                           | 10       |
| 4.3.3      | Sequence diagram                           | 10       |
| 4.3.4      | Collaboration Diagram                      | 11       |
| 4.3.5      | Activity Diagram                           | 11       |
| 4.4        | Data dictionary                            | 12       |
| CHAPTER 5: | IMPLEMENTATION                             | 13       |
| 5.1        | Technology used                            | 13       |
| 5.2        | Frontend designs                           | 15       |
| CHAPTER 6: | TESTING                                    | 31       |
| 6.1        | VPC creation test                          | 31       |
| 6.2        | Subnet creation test                       | 31       |
| 6.3        | IGW test                                   | 32       |
| 6.4        | Route tables test                          | 32       |
| 6.5        | Webserver and database test                | 33       |
| 6.6        | Jump box test                              | 34       |
| 6.7        | NAT gateway test                           | 34       |

|            |                             |    |
|------------|-----------------------------|----|
| CHAPTER 7: | RESULTS                     | 35 |
| CHAPTER 8: | CONCLUSION                  | 36 |
| CHAPTER 9: | FUTURE SCOPE & ENHANCEMENTS | 37 |
|            | BIBLIOGRAPHY                | 38 |

## ACKNOWLEDGEMENT

The success of any course depends mostly on the teachers who teach us. Only good teaching can interpret the syllabus and produce desirable changes and competent citizens. This one was a team effort and many people whose names do not appear on the cover deserve credit. First, we thank God almighty for his manifold mercies in carrying out of our project successfully.

We heartfully thank our principal **Dr. K. ESHWARIAH**, for providing all the resources for completing our project.

We sincerely thank our Head of the Department of Computer Science & Engineering and Professor **Dr. Major. RAVINDRA BABU KALLAM**, for encouraging us in doing real-time projects and for his guidance.

We thank our project coordinator **Mrs. RADARAPU BHARATHI**, Assistant Professor in CSE for their continuous support in completion of this project.

We thank our internal guide **Mrs. FATHIMA ANJUM**, Assistant Professor in the department of Computer Science & Engineering, for her guidance and support.

We also thank other teaching and non-teaching staff of Kamala Institute of Technology and Science, for supporting us in every stage of our mini project, entitled “**AWS VPC ARCHITECTURE WITH TWO SUBNETS, ROUTE TABLES, AND NAT GATEWAY**”. We also thank our parents and friends for their moral support throughout the project work that helped to strengthen our will.

|                             |                     |
|-----------------------------|---------------------|
| <b>PITTALA HARITHA</b>      | <b>(22281A05E7)</b> |
| <b>PONNOJU ASHWITHA</b>     | <b>(22281A05I4)</b> |
| <b>YELLENKI RUSHITHA</b>    | <b>(22281A05I7)</b> |
| <b>GUNDUKONDA PRAVALIKA</b> | <b>(22281A05D3)</b> |

## **ABSTRACT**

Amazon Virtual Private Cloud (VPC) provides a logically isolated environment within the AWS cloud. It enables users to define custom IP address ranges and configure subnets and route tables. VPCs typically include both public and private subnets to separate external-facing and internal resources. Public subnets connect to the internet through an Internet Gateway. Private subnets can access the internet securely via a NAT Gateway in the public subnet. Security Groups act as virtual firewalls, controlling inbound and outbound instance-level traffic. Elastic IPs and Auto Scaling can be used to support high availability and resilience. VPC flow logs help monitor and troubleshoot traffic.

## LIST OF FIGURES:

| Figure No. | Name of the Figure      | Page no. |
|------------|-------------------------|----------|
| 4.1        | Block Diagram           | 8        |
| 4.2        | DFD(Data flow diagram)  | 9        |
| 4.3        | Class Diagram           | 9        |
| 4.4        | Use case diagram        | 10       |
| 4.4.1      | Sequence diagram        | 10       |
| 4.4.2      | Collaboration diagram   | 11       |
| 4.4.3      | Activity diagram        | 11       |
| 5.2.1      | Creating a VPC          | 15       |
| 5.2.2      | Public subnet creation  | 15       |
| 5.2.3      | Private subnet Creation | 16       |
| 5.2.4      | subnets                 | 16       |
| 5.2.5      | IGW (Internet gateway)  | 17       |
| 5.2.6      | VPC Attach              | 17       |
| 5.2.7      | Route Creation          | 18       |
| 5.2.8      | Subnet association      | 18       |
| 5.2.9      | Edit routes             | 18       |
| 5.2.10     | Routes                  | 19       |
| 5.2.11     | Webserver               | 19       |
| 5.2.12     | Network settings        | 20       |
| 5.2.13     | Inbound Rules           | 20       |
| 5.2.14     | Bin bash commands       | 21       |
| 5.2.15     | Bin bash commands       | 21       |
| 5.2.16     | Launch instances        | 22       |
| 5.2.17     | webserver address       | 22       |

|        |                        |    |
|--------|------------------------|----|
| 5.2.18 | Static website         | 23 |
| 5.2.19 | Database               | 23 |
| 5.2.20 | Network settings       | 24 |
| 5.2.21 | Inbound Rules          | 24 |
| 5.2.22 | Database server        | 25 |
| 5.2.23 | Jump box               | 25 |
| 5.2.24 | Jump box               | 26 |
| 5.2.25 | Jump box address       | 26 |
| 5.2.26 | SSH client             | 27 |
| 5.2.27 | Putty                  | 27 |
| 5.2.28 | Putty                  | 28 |
| 5.2.29 | WinSCP                 | 28 |
| 5.2.30 | WinSCP                 | 29 |
| 5.2.31 | WinSCP                 | 29 |
| 5.2.32 | NAT Gate               | 30 |
| 5.2.33 | Putty                  | 30 |
| 6.1    | Testing VPC            | 31 |
| 6.2    | Testing Subnet         | 31 |
| 6.3    | Testing IGW            | 32 |
| 6.4    | Testing Routes         | 32 |
| 6.5    | Webserver and database | 33 |
| 6.6    | Jump box               | 34 |
| 6.7    | NAT gateway            | 34 |
| 7      | Website                | 35 |



# CHAPTER 1

## INTRODUCTION

### 1.1 About the project

This project focuses on the design and implementation of Virtual Private Cloud (VPC) architecture in Amazon Web Services (AWS) to support secure, scalable, and high-performance application deployment. The main objective is to create a logically isolated network environment where cloud resources such as EC2 instances, RDS databases, and load balancers can operate securely and efficiently.

The architecture follows a multi-tier model, dividing resources into public and private subnets across multiple Availability Zones (AZs) to ensure high availability and fault tolerance. Internet Gateways and NAT Gateways are used to manage external access, while Security Groups and Network ACLs enforce strict traffic control and security.

This VPC setup supports real-world enterprise use cases such as hosting web applications, internal services, and databases with proper isolation, redundancy, and monitoring. The infrastructure is designed for scalability, security, and compliance, leveraging native AWS services like CloudWatch for monitoring and IAM for access control.

### 1.2 Drawbacks of Existing VPC Architecture

#### 1. Complexity in Network Management:

- Managing multiple subnets, route tables, NACLs, and security groups can become very complex.
- Misconfiguration can expose services unintentionally.

#### 2. High Cost:

- NAT Gateways are expensive when handling high volumes of traffic.
- Multiple AZ deployments increase cost for high availability.

#### 3. Scalability Limitations:

- IP address exhaustion can occur if subnet sizing isn't planned properly.
- Limited number of VPCs per region (by default) can restrict growth.

#### 4. Latency in Cross-AZ Communication:

- Data transfer between AZs, while redundant, introduces additional latency and cost.

#### 5. Limited Traffic Visibility:

- Without advanced logging, it's hard to trace traffic flows for debugging or security auditing.

#### 6. Security Risks:

- Relying heavily on Security Groups/NACLs may lead to misconfigured access rules.
- Bastion host exposure adds a potential attack surface if not properly secured.

## 1.3 Proposed system with features

### 1. Custom VPC Design:

- CIDR block configuration (e.g., 10.0.0.0/16).
- Logical network isolation for enhanced control and security.

### 2. Multi-Tier Subnet Architecture:

- Public Subnets: Host load balancers, bastion host, and NAT Gateways.
- Private Subnets: Host application servers, databases, and internal services.
- Deployed across multiple Availability Zones (AZs) for high availability.

### 3. Internet Gateway & NAT Gateway:

- Internet Gateway: Enables internet access for public subnet resources.
- NAT Gateway: Allows private subnet resources to access the internet securely.

### 4. Security Controls:

- Security Groups: Instance-level firewalls to control inbound/outbound traffic.
- Network ACLs: Subnet-level stateless firewall rules.
- IAM Roles and Policies: Granular access control to AWS services.

### 5. High Availability & Fault Tolerance:

- Resource distribution across at least two AZs.
- Auto Scaling Group for EC2 instances.
- Multi-AZ RDS deployment for database redundancy.

### 6. Monitoring & Logging:

- Amazon CloudWatch: For metrics, logs, and alerts.
- VPC Flow Logs: To monitor network traffic.
- AWS CloudTrail: To track user activity and API calls.

### 7. Secure Administration:

- Bastion Host in public subnet for controlled SSH access to private instances.
- Use of Key Pairs for secure logins.

### 8. Elastic Load Balancing (ELB):

- Distributes incoming traffic across multiple EC2 instances in different AZs.
- Increase fault tolerance and application availability.

### 9. DNS and Traffic Management:

- Amazon Route 53 for DNS resolution and traffic routing.
- Health checks and failover configuration.

### 10. Cost Optimization Strategies:

- Use of Spot Instances and Auto Scaling to manage costs.
- Rightsizing resources and implementing budgets/alerts.

## **CHAPTER 2**

### **LITERATURE SURVEY**

#### **1. Introduction**

With the rise of cloud computing, enterprises have transitioned from traditional data centers to cloud-based infrastructures to achieve scalability, cost-efficiency, and faster deployments. One of the foundational components in cloud architecture is the Virtual Private Cloud (VPC). In AWS, VPC allows users to create isolated network environments where they can launch and manage cloud resources securely.

#### **2. Evolution and Purpose of VPC**

Originally introduced to mimic on-premises data center environments in the cloud, AWS VPC has evolved into a critical element of cloud network design[3]. It enables full control over IP address ranges, subnets, routing tables, and gateways. According to the AWS Well-Architected Framework, implementing a structured VPC design helps organizations maintain strong security, performance, and fault tolerance [1].

#### **3. Key Components of AWS VPC Architecture**

- Literature from both academic and industry sources outlines the following core components:
- Subnets: Divide the VPC into public and private zones, aiding in traffic control and resource isolation[1].
- Route Tables: Define how traffic is directed between subnets and the internet[2].
- Internet Gateway (IGW) and NAT Gateway: Used for internet access by public and private resources respectively.
- Security Groups (SGs) and Network ACLs (NACLs): Provide layered security, filtering inbound/outbound traffic.
- Elastic Load Balancing (ELB): Distributes traffic across multiple instances for high availability.
- VPC Peering & Transit Gateway: Enable communication between multiple VPCs and facilitate centralized networking [1].

#### **4. Security Practices and Literature Insights**

- Security within a VPC is a major focus area. Research by Mavridis et al. (2020) highlights the effectiveness of combining security groups (stateful firewalls) with network ACLs (stateless filters) for a multi-layered defense. Additionally, VPC endpoints have been widely discussed as a method for accessing AWS services (e.g., S3) without traversing the public internet, thereby reducing risk exposure.
- The integration of Identity and Access Management (IAM) with VPC-based resources helps enforce the principle of least privilege, as recommended by NIST cybersecurity frameworks [2].

#### **5. High Availability and Multi-AZ Deployments**

According to studies in cloud infrastructure design (Hwang & Li, 2018), deploying VPCs across multiple Availability Zones (AZs) increases fault tolerance and application uptime. AWS services such as Auto Scaling, Multi-AZ RDS, and Elastic Load Balancer (ELB) [1] are commonly used within VPCs to maintain service continuity during failures or traffic spikes.

## **6. Monitoring and Observability**

Monitoring is essential for ensuring performance and compliance. The literature agrees that AWS native tools like Cloud Watch, VPC Flow Logs, and Cloud Trail offer deep visibility into traffic patterns, instance behavior, and configuration changes.

## **7. Automation and Infrastructure as Code (IAC)**

Modern VPC deployments leverage Infrastructure as Code (IAC) tools such as Terraform, AWS Cloud Formation, to automate setup and management. These tools help reduce configuration errors, support CI/CD pipelines, and make deployments reproducible. Researchers note that automated VPC provisioning is now the best practice in DevOps culture[3].

## **8. Limitations of Traditional and Current VPC Approaches**

- Despite its benefits, AWS VPC architecture has some limitations:
- Complexity: Designing VPCs at scale with hundreds of subnets, routes, and firewalls can be difficult.
- Cost Management: NAT gateways, data transfer fees, and private link usage can significantly increase operational costs.
- IP Address Planning: Poor subnet sizing can lead to IP exhaustion, especially in large environments[2].
- Operational Overhead: Debugging connectivity issues or misconfigured routes can be time-consuming without deep cloud networking expertise.

## **9. Comparative Studies**

Studies comparing AWS VPC with Google Cloud VPC and Azure Virtual Network reveal that while AWS offers more granular control and mature networking features[3], its complexity can be higher. For example, AWS allows fine-grained subnet routing and more flexible peering options, whereas Azure emphasizes ease of use[1].

## **10. Emerging Trends**

- Zero Trust Networking: Integration of identity-driven access control within VPCs.
- Serverless VPC Access: Lambda functions now support VPC access, enabling hybrid deployments without traditional computers.
- VPC Lattice (new AWS feature): Provides application-layer networking across services, simplifying service-to-service communication.
- Centralized Networking: Use of AWS Transit Gateway for managing inter-VPC communication at scale [2].

## CHAPTER 3

### ANALYSIS

#### 3.1 Hardware and Software Requirements

##### 1. Hardware Requirements:

Since AWS VPC is a cloud-based virtual network, it does not require traditional on-premises hardware for deployment. However, to interact with AWS and manage the VPC, the following client-side hardware is recommended:

- Processor-Intel i5/i7 or AMD Ryzen 5 or better
- RAM-Minimum 8 GB (16 GB recommended for smooth multitasking)
- Storage-Minimum 256 GB SSD (HDD acceptable for light usage)
- Internet-High-speed broadband internet (minimum 10 Mbps)
- Display-1080p resolution or higher for better UI experience

##### 2. Software Requirements:

- Operating System: Windows 10/11, mac OS, or Linux (Ubuntu/CentOS)  
To run the AWS CLI, SDKs, and infrastructure tools.
- Web Browser: Chrome, Firefox, Edge, or Safari (latest versions)  
To access the AWS Management Console
- AWS CLI: AWS Command Line Interface (CLI) v2  
For terminal-based interaction with AWS resources
- SDKs (optional): AWS SDKs for Python (Boto3), Java, Node.js, etc.  
For programmatic access and automation
- IAC Tools: Terraform, AWS Cloud Formation, or Pulumi  
For infrastructure automation and provisioning
- Network Tools: PUTTY, Open SSH, Postman, Wireshark (optional) For SSH, API testing, and traffic monitoring
- Monitoring Tools: AWS Cloud Watch, Cloud Trail, VPC Flow Logs (AWS-native)  
For logging, monitoring, and troubleshooting
- Version Control: Git, GitHub/Git Lab  
For managing infrastructure-as-code repositories

#### 3.2 Functional and Non-Functional Requirements

##### Functional Requirements:

###### 1.VPC Creation and Subnet Management:

It is the ability to create a Virtual Private Cloud (VPC) with customizable IP address ranges. Support for creating multiple public and private subnets within the VPC.

###### 2.Routing and Traffic Control:

Configuration of route tables for directing traffic between VPC subnets, the internet, and external resources. Integration with Internet Gateway (IGW) for internet access and NAT Gateway for private subnet access.

### 3.Security and Access Control:

Use Security Groups to manage inbound and outbound traffic at the instance level. Setup of Network ACLs to filter traffic at the subnet level for additional security layers.

### 4.High Availability and Load Balancing:

Deployment of resources across multiple Availability Zones (AZs) to ensure redundancy and fault tolerance. Integration of Elastic Load Balancer (ELB) to distribute traffic evenly across instances.

### 5.Monitoring and Logging:

Enablement of VPC Flow Logs for real-time network traffic monitoring. Integration with CloudWatch and CloudTrail to monitor, audit, and alert for any changes in the VPC.

## **Non-Functional Requirements:**

### 1.Scalability:

The VPC must support horizontal scaling, allowing the addition of more instances, subnets, or resources without performance degradation.

### 2.Availability and Fault Tolerance:

The architecture must provide high availability by distributing resources across multiple Availability Zones and employing automatic recovery mechanisms.

### 3.Security and Compliance:

The VPC must comply with industry security standards, ensuring that data encryption, least privileged access, and audit logs are enabled for regulatory compliance (e.g., GDPR, HIPAA).

### 4.Performance:

The VPC should ensure low-latency network communication between resources and offer high throughput for applications with demanding network requirements.

### 5.Cost Efficiency:

The VPC architecture should minimize costs by optimizing the use of AWS services (e.g., NAT Gateway management, Elastic IP allocation) and preventing over-provisioning of resources.

## **3.3 Module Description:**

The AWS VPC Architecture project is divided into several core modules, each designed to manage a specific functionality within the Virtual Private Cloud environment. These modules work together to deliver a secure, scalable, and efficient cloud infrastructure that meets both operational and business requirements.

The first module is the VPC Setup Module, which focuses on the creation and basic configuration of the Virtual Private Cloud. This includes defining a custom IP address range using CIDR blocks, enabling IPv4 support (and optionally IPv6), and configuring DNS support and hostnames. It also involves establishing custom route tables that determine how traffic is routed within the VPC.

This module lays the foundation for all other networking components and ensures the VPC is correctly structured. Subnet Management Module, which organizes the network into smaller, isolated segments known as subnets. These subnets are classified into public and private subnets based on their accessibility.

Public subnets are configured to allow communication with the internet, while private subnets are used for internal resources. Subnets are deployed across multiple Availability Zones to ensure redundancy and high availability.

Each subnet is also associated with specific route tables and network configurations to control traffic flow effectively.

The Internet and NAT Gateway Module is responsible for managing connectivity between the VPC and the internet. It involves the creation and attachment of an Internet Gateway (IGW) for enabling internet access to public-facing instances.

For private subnets, a NAT Gateway is set up to provide secure outbound internet access, preventing unsolicited inbound traffic while allowing instances to download updates or connect to external services. Elastic IP addresses are also managed within this module to ensure stable public IP configurations.

## CHAPTER 4

### DESIGN

#### 4.1 Block Diagram:

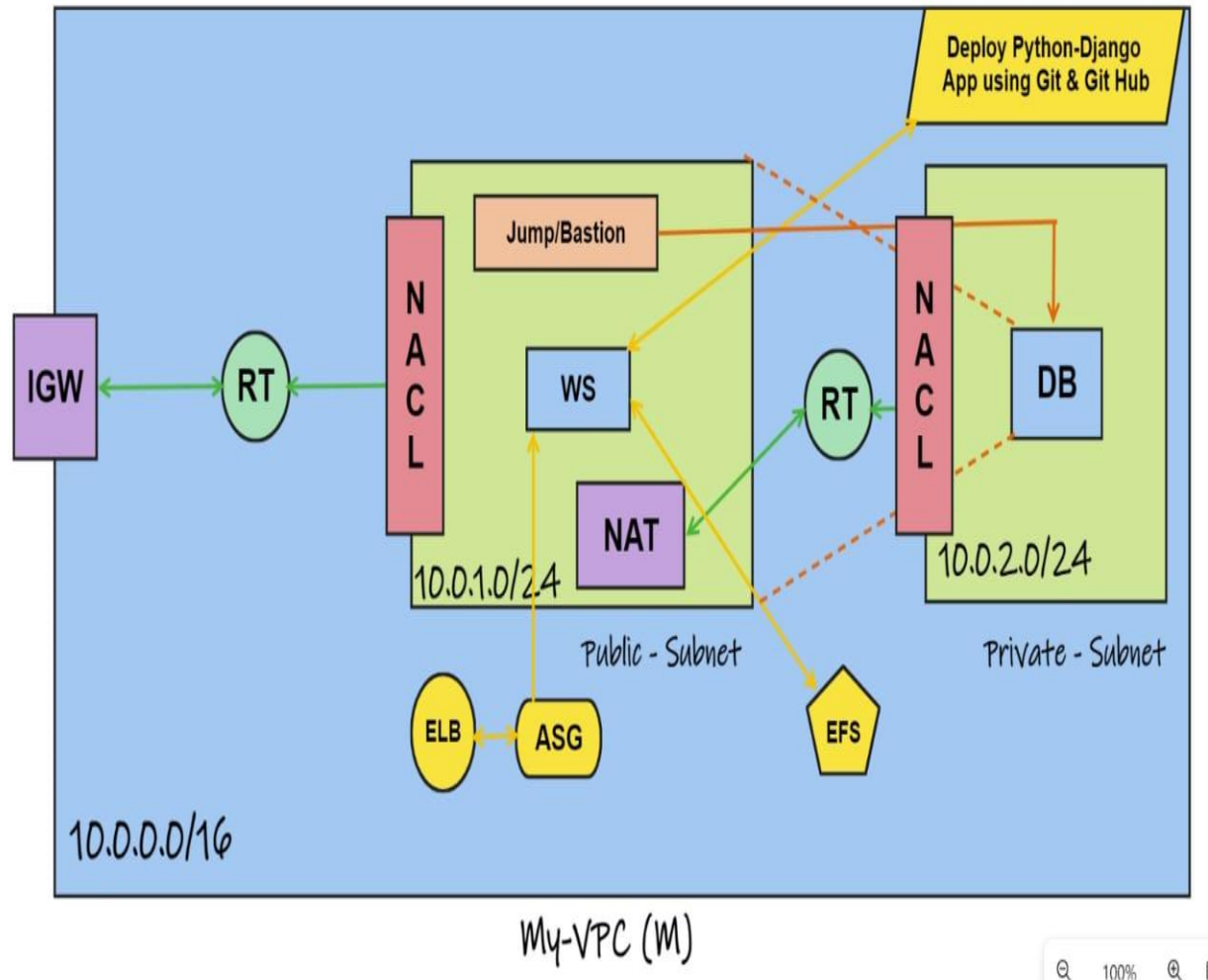


Figure 4.1: AWS VPC Architecture Block Diagram

The block diagram of VPC architecture visually represents its isolated network environment in AWS, showcasing key components like subnets, route tables, gateways, and security layers are shown in the above figure 4.1. It illustrates how these elements interact to enable secure and scalable resource deployment within the virtual private cloud .



## 4.2 DFD (Data flow diagram):

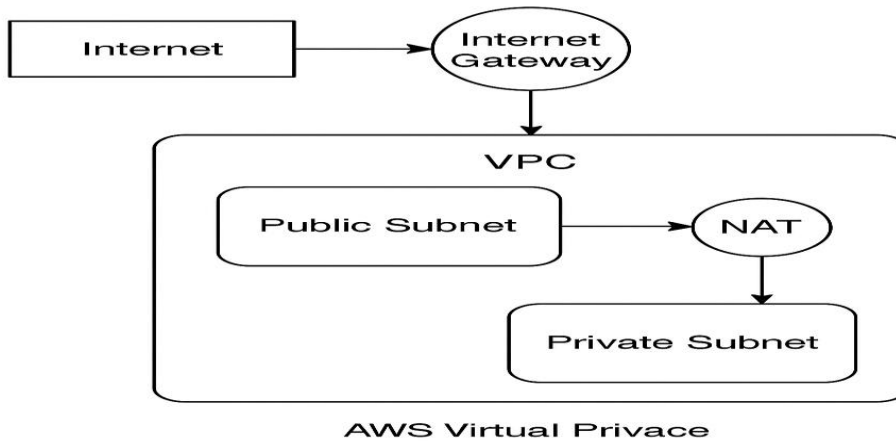


Figure 4.2: Data flow diagram

Data Flow Diagram (DFD) to an AWS Virtual Private Cloud (VPC) architecture provides a visual and logical representation. as shown in the above figure 4.2 the data moves through your isolated cloud network Instead of focusing solely on the infrastructure components.

**4.3 UML Diagrams:** UML (Unified Modeling Language) diagrams visually represent the structure and behavior of a system using standardized symbols. They help in designing, analyzing, and documenting software architecture and workflows.

### 4.3.1 Class Diagram:

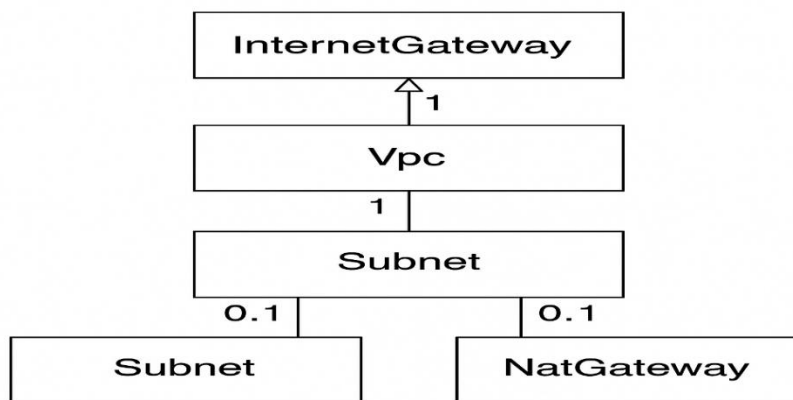


Figure 4.3.1: Class diagram

Class diagram of VPC aggregates Subnets, Route Tables, Security Groups as shown in the above figure 4.3.1 Subnets contain Network Interfaces, and Route Tables manage Routes to Gateways.

### 4.3.2 Use case Diagram:

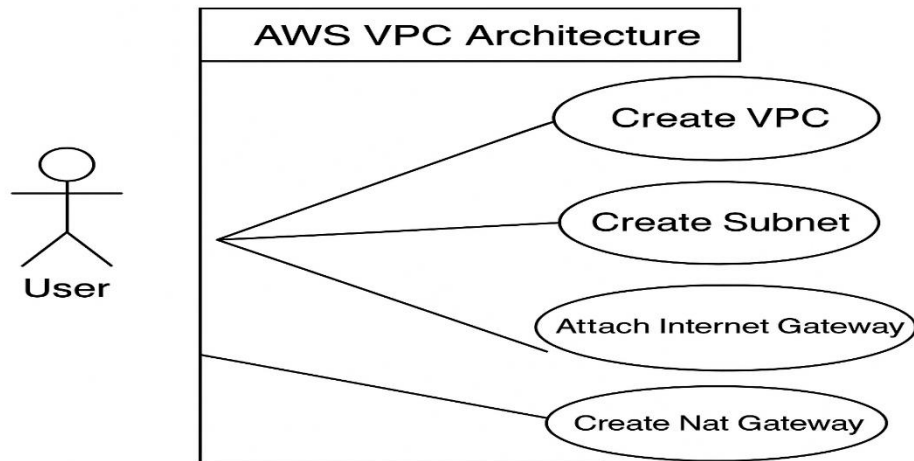


Figure 4.3.2: use case diagram

Use case diagram in this the User Creates and Manages VPC to establish an isolated network. As shown in the above figure 4.3.2 User Control Instance Access using Security Groups. Users Establish Connectivity via Internet Gateways or NAT Gateways. Finally, they Organize Resources within Subnets and manage traffic with Route Table.

### 4.3.3 Sequence Diagram:

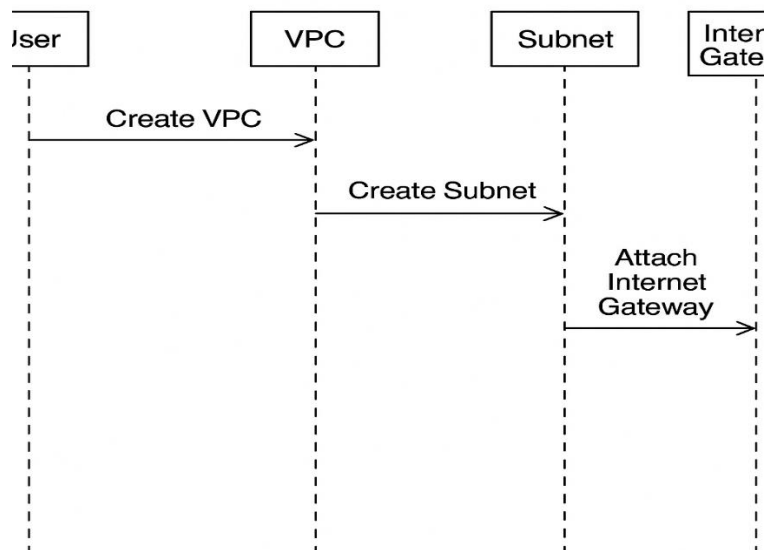


Figure 4.3.3: Sequence Diagram

The User request triggers the VPC service to provision core components like the VPC and subnets. The above figure 4.3.3 shows Subsequently, the user configures Route Tables and Gateways to direct traffic. Finally, the user defines Security Group rules which are enforced during instance communication.

#### 4.3.4 Collaboration Diagram:

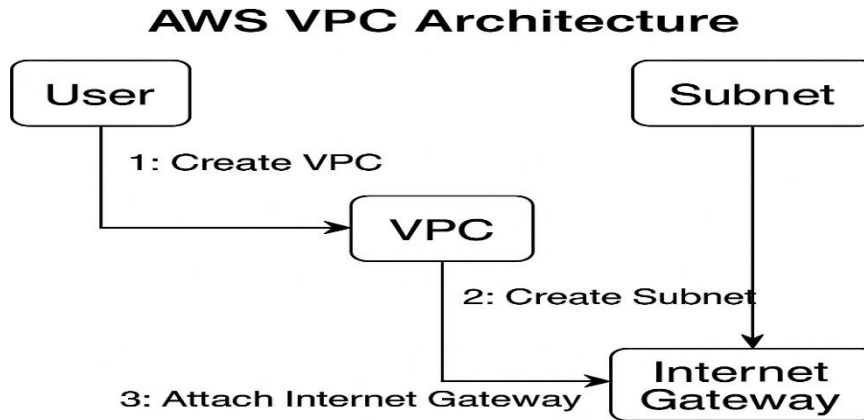


Figure 4.3.4: Collaboration diagram

That the User initiates VPC setup, leading the VPC Service to coordinate the creation of the VPC and Subnets. The above figure 4.3.4 shows Network paths are established through the interaction of Route Tables and Gateways. Access control for instances is managed collaboratively by the VPC Service and Security Groups associated with their Network Interfaces.

#### 4.3.5 Activity Diagram:

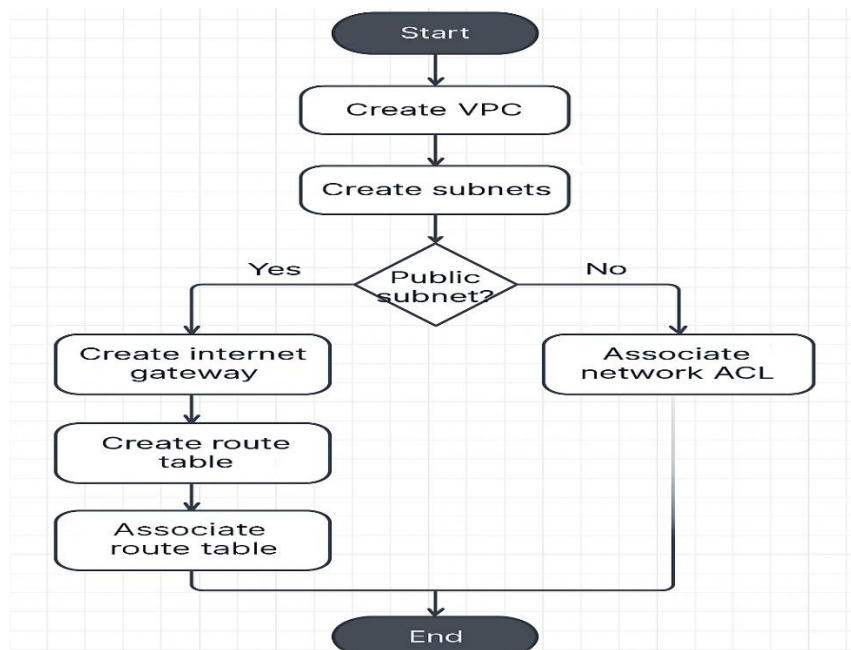


Figure 4.3.5: Activity diagram

The activity diagram for AWS VPC architecture would primarily show the creation and configuration of VPC components like subnets, route tables, and gateways, The above figure 4.3.5 shows the launching and interaction of resources within this defined virtual network.

#### 4.4 Data Dictionary:

The AWS Private Cloud) architecture consists of several foundational components, each with distinct attributes. The VPC is the primary network container, identified by a VPC id and defined by a CIDR block representing its IP range. It includes properties like instance tenancy (either default or dedicated) and tags for resource identification. Within the VPC, subnets are created, each with a subnet id, CIDR block, and availability zone. Subnets may also be configured to automatically assign public IPs to launched instances using the map public IP on launch flag.

Traffic within the VPC is managed by Route Tables, which include routing rules (routes) to direct network traffic. Each route table is associated with one or more subnets and includes a route table id, the VPC id, and a flag to indicate whether it is the main route table. For external connectivity, an Internet Gateway (IGW) is used to allow internet access to resources within the VPC. The IGW is identified by an internet gateway id and is attached to a specific VPC.

For enabling internet access from private subnets, a NAT Gateway is commonly deployed. It is associated with a NAT gateway id, resides in a public subnet, and uses an Elastic IP, referenced by its allocation id. Security Groups act as virtual firewalls for instance, controlling inbound and outbound traffic. Each security group includes a security group id, group name, description, rules for allowed traffic, and is tied to a specific VPC.

Additionally, Elastic IPs (EIPs) are static public IP addresses allocated to AWS resources like NAT Gateways or EC2 instances. Each EIP is identified by an allocation id and a public Ip. For security, private communication between VPCs, VPC Peering Connections can be established. These connections are defined by a VPC peering and include the Requester VPC id acceptor VPC id, and a status to indicate connection state. All these components support tagging for better organization and resource management.

## CHAPTER 5

### IMPLEMENTATION

#### 5.1 Technology Used:

##### 1. Software-Defined Networking (SDN):

- Enables programmable control over the network via AWS APIs.
- Manages subnets, route tables, gateways, and network interfaces dynamically.

##### 2. Core Networking Components:

- Subnets: Logical subdivisions of a VPC's IP range.
- Route Tables: Control the routing of traffic within the VPC.
- Internet Gateway (IGW): Enables internet access for instances in the VPC.
- NAT Gateway/Instance: Allows private subnets to access the internet.
- Elastic Network Interfaces (ENI): Virtual network interfaces for EC2 instances.

##### 3. Connectivity Technologies

###### VPC Peering:

- Connects two VPCs for internal communication.
- AWS Transit Gateway: Central hub for connecting multiple VPCs and on-prem networks.
- AWS Direct Connect: Provides a dedicated physical connection to AWS.
- AWS VPN: Secure connection between on-premises networks and AWS VPC.

##### 4. IP Address Management

###### CIDR Blocks:

- Define the IP range of the VPC and its subnets.
- Private IPs & Elastic IPs: Used for internal and external communication.

##### 5. Security Technologies:

- Instance-level virtual firewalls.
- Network ACLs: Subnet-level firewalls for inbound/outbound rules.
- AWS IAM (Identity and Access Management): Controls access to VPC resources.

##### 6. Infrastructure Automation Tools AWS Cloud Formation: AWS-native infrastructure as code tool.

- Terraform: Third-party IAC tool widely used for VPC automation.
- AWS CDK: Allows defining infrastructure using programming languages.

## 7. Monitoring & Logging

- VPC Flow Logs: Captures information about IP traffic in your VPC.
- Amazon Cloud Watch: For metrics and alerting.
- AWS Cloud Trail: Logs all API activity within the VPC.

## 8. DNS & Name Resolution

- Amazon Route 53: DNS service that works with public and private domains in your VPC.

## 9. VPC Flow Logs

- Purpose: Captures detailed information about IP traffic going to and from network interfaces in your VPC.
- Use Cases: Security analysis, performance monitoring, and troubleshooting.
- Integration: Logs can be sent to Amazon CloudWatch or S3.

## 10. AWS Private Link

- Purpose: Provides private connectivity to services across VPCs or AWS accounts without using public IPs.
- Use Cases: Exposing services securely without exposing them to the internet.
- Components: Endpoint services, interface endpoints.

## 11. Elastic Load Balancing (ELB)

- Types: Application Load Balancer (ALB), Network Load Balancer (NLB), Gateway Load Balancer (GWLb).
- Purpose: Distributes traffic among multiple targets (EC2, containers, IPs) within VPC.
- Security: Works with Security Groups and supports VPC integration.

## 5.2 Frontend design:

### 5.2.1 Creating a VPC:

[VPCs](#) > Create VPC

### Create VPC [Info](#)

A VPC is an isolated portion of the AWS Cloud populated by AWS objects, such as Amazon EC2 instances.

#### VPC settings

**Resources to create** [Info](#)  
Create only the VPC resource or the VPC and other networking resources.

☒ VPC only ☐ VPC and more

**Name tag - optional**  
Creates a tag with a key of 'Name' and a value that you specify.

my-vpc

**IPv4 CIDR block** [Info](#)  
☒ IPv4 CIDR manual input  
☐ IPAM-allocated IPv4 CIDR block

**IPv4 CIDR**  
10.0.0.0/16  
CIDR block size must be between /16 and /28.

**IPv6 CIDR block** [Info](#)  
☒ No IPv6 CIDR block  
☐ IPAM-allocated IPv6 CIDR block  
☐ Amazon-provided IPv6 CIDR block  
☐ IPv6 CIDR owned by me

**Tenancy** [Info](#)  
Default

Figure:5.2.1: Creating a VPC

The above figure 5.2.1 shows creation of virtual private cloud with tag and IPv4 CIDR range.

### 5.2.2 Public subnet creation:

[Subnets](#) > Create subnet

### Create subnet [Info](#)

**VPC**  
**VPC ID**  
Create subnets in this VPC.

vpc-0864541de209bc08e (my-vpc)

**Associated VPC CIDRs**  
IPv4 CIDRs  
10.0.0.0/16

#### Subnet settings

Specify the CIDR blocks and Availability Zone for the subnet.

**Subnet 1 of 2**

**Subnet name**  
Create a tag with a key of 'Name' and a value that you specify.

my-pubsubnet  
The name can be up to 256 characters long.

**Availability Zone** [Info](#)  
Choose the zone in which your subnet will reside, or let Amazon choose one for you.

Asia Pacific (Mumbai) / ap-south-1a

**IPv4 VPC CIDR block** [Info](#)  
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

10.0.0.0/16

**IPv4 subnet CIDR block**

Figure 5.2.2: public subnet creation

The above figure 5.2.2 shows subnets are created by using tag CIDR range and availability zones.

### 5.2.3 Private subnet creation:

The screenshot shows the 'Subnet 2 of 2' creation form in the AWS console. It includes fields for 'Subnet name' (my-prvsubnet), 'Availability Zone' (Asia Pacific (Mumbai) / ap-south-1b), 'IPv4 VPC CIDR block' (10.0.0/16), and 'IPv4 subnet CIDR block' (10.0.2.0/24). There is a 'Tags - optional' section with a key-value pair (Name: my-prvsubnet). At the bottom, there are 'Cancel' and 'Create subnet' buttons.

Figure 5.2.3: Private subnet creation.

The above figure 5.2.3 shows Creation of subnets with tag, availability zones, subnet CIDR block and click create subnet.

### 5.2.4 Subnets:

The screenshot shows the 'Subnets' page in the AWS console. A green notification bar at the top states: 'You have successfully created 2 subnets: subnet-0556c3a26c5862914, subnet-02fe0a0fd1f4d9d8e'. Below this, there is a table of subnets. The table has columns: Name, Subnet ID, State, VPC, Block Public..., IPv4 CIDR, IPv6 CIDR, and IPv6 C. Two subnets are listed: 'my-pubsubnet' and 'my-prvsubnet', both in 'Available' state.

| Name         | Subnet ID                | State     | VPC                            | Block Public... | IPv4 CIDR   | IPv6 CIDR | IPv6 C |
|--------------|--------------------------|-----------|--------------------------------|-----------------|-------------|-----------|--------|
| my-pubsubnet | subnet-0556c3a26c5862914 | Available | vpc-0864541de209bc08e   my-... | Off             | 10.0.1.0/24 | -         | -      |
| my-prvsubnet | subnet-02fe0a0fd1f4d9d8e | Available | vpc-0864541de209bc08e   my-... | Off             | 10.0.2.0/24 | -         | -      |

Figure 5.2.4: Subnets

The above figure 5.2.4 shows Subnets are created, tap on public subnet go to actions select edit subnet settings enable Auto scaling.



## 5.2.5 Internet Gateway:

The screenshot shows the 'Create internet gateway' page in the AWS Management Console for the 'ap-south-1' region. The page title is 'Create internet gateway' with an 'Info' icon. A sub-header states: 'An internet gateway is a virtual router that connects a VPC to the internet. To create a new internet gateway specify the name for the gateway below.' The main form is titled 'Internet gateway settings'. It contains a 'Name tag' section with a text input field containing 'my-igw'. Below this is a 'Tags - optional' section. It explains that a tag is a label with a key and an optional value. There is a 'Key' input field with 'Name' and a 'Value - optional' input field with 'my-igw'. A 'Remove' button is next to the value field. An 'Add new tag' button is at the bottom of the tags section. At the bottom right of the form are 'Cancel' and 'Create internet gateway' buttons.

Figure 5.2.5: Internet Gateway

The above figure 5.2.5 shows Creation of internet gateway by providing a name tag and click on create internet gateway.

## 5.2.6 VPC Attach:

The screenshot shows the 'Internet gateway' page in the AWS Management Console. A green notification banner at the top states: 'Internet gateway igw-0fa379d49e56df082 successfully attached to vpc-0864541de209bc08e'. The page title is 'igw-0fa379d49e56df082 / my-igw'. The 'Details' section shows the 'Internet gateway ID' as 'igw-0fa379d49e56df082', the 'State' as 'Attached', the 'VPC ID' as 'vpc-0864541de209bc08e | my-vpc', and the 'Owner' as '464858727173'. The 'Tags' section shows a search bar and a table with one tag: 'Name' with value 'my-igw'. A 'Manage tags' button is next to the tags section. The left sidebar shows the 'VPC dashboard' with a 'Filter by VPC' dropdown and a list of VPC resources including 'Virtual private cloud', 'Your VPCs', 'Subnets', 'Route tables', 'Internet gateways', 'Egress-only internet gateways', 'DHCP option sets', 'Elastic IPs', 'Managed prefix lists', 'NAT gateways', 'Peering connections', and 'Security'.

Figure 5.2.6: VPC Attach

The above figure 5.2.6 shows Internet gate way is created tap on attach to a VPC it is right side on the screen in green color.

## 5.2.7 Route Creation:

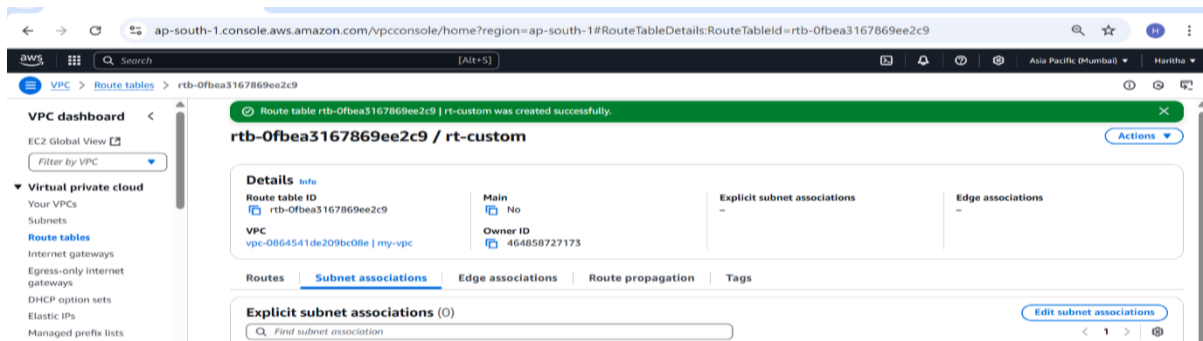


Figure 5.2.7: Route creation

The above figure 5.2.7 shows Route tables creation to determine how network traffic is routed within and outside the VPC and route-main and route-custom are created.

## 5.2.8 Subnet Association:

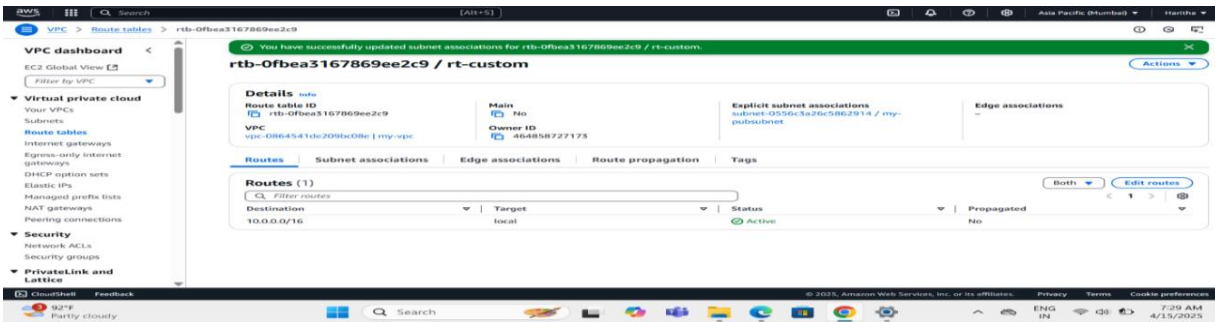


Figure 5.2.8: Subnet Association

The above figure 5.2.8 shows subnet associations are updated successfully.

## 5.2.9 Edit routes:

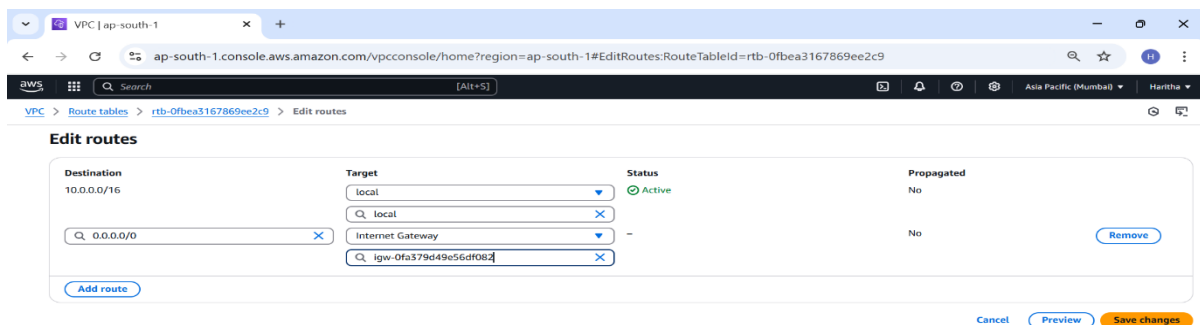


Figure 5.2.9: Edit Routes

The above figure 5.2.9 shows about editing the routes by giving destination address and Target to save changes.

## 5.2.10 Routes:

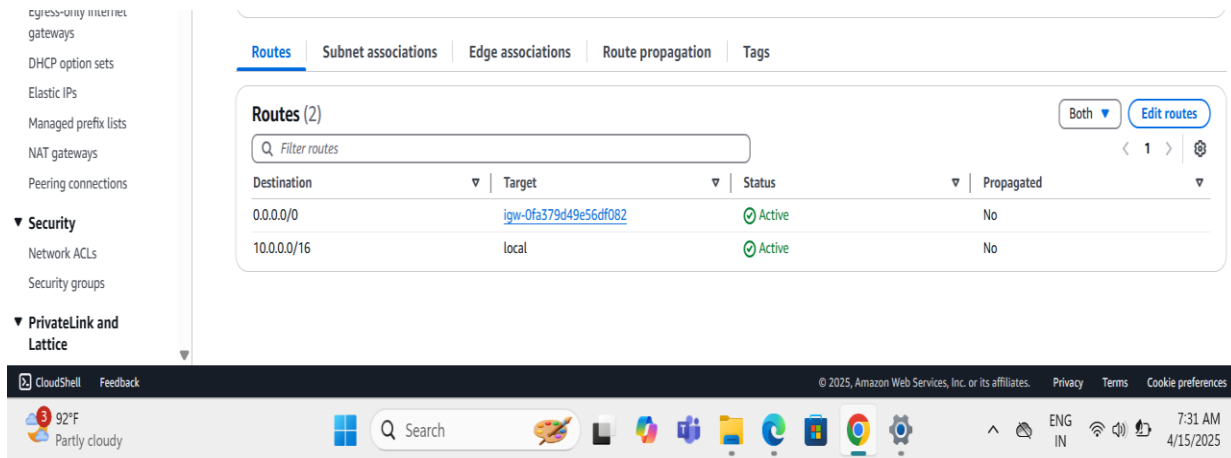


Figure 5.2.10: Routes

The above figure 5.2.10 shows that Two route tables are created route-main and route-custom after that go to subnet associations tap on public subnet click save associate next, we need to edit the routes.

## 5.2.11 Webserver:

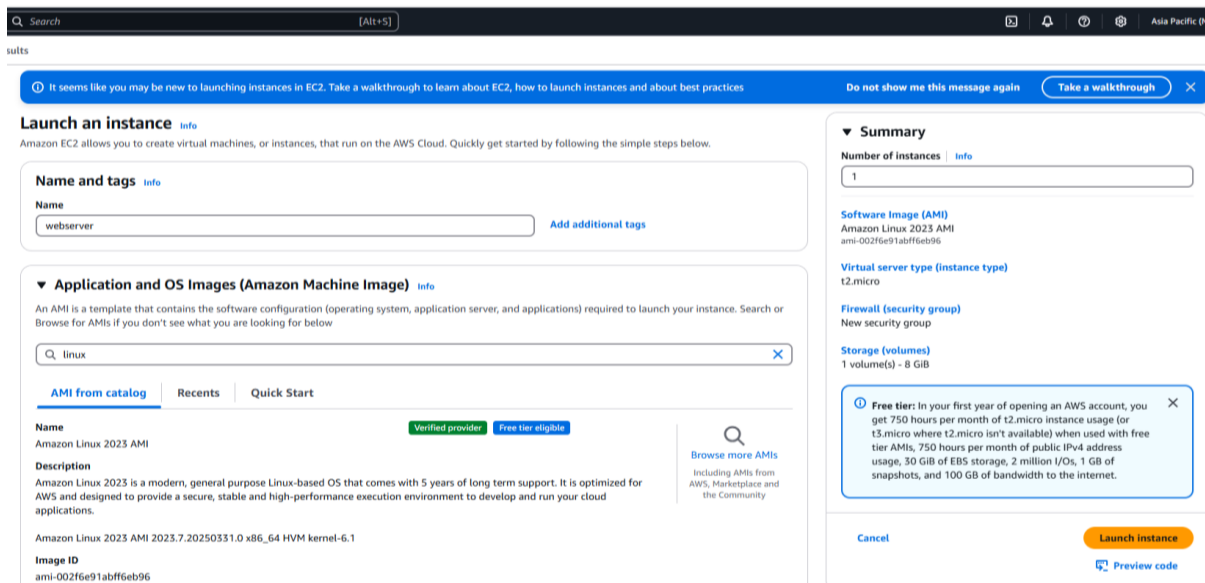


Figure 5.2.11: Webserver

The above figure 5.2.11 shows webserver is created by giving tag and Linux OS which is helpful to connection in VPC

## 5.2.12 Network settings:

The screenshot shows the 'Launch Instance' page in the AWS Management Console, specifically the 'Network settings' tab. The page is for the 'ap-south-1' region. The 'Key pair name' is set to 'web'. The 'VPC' is 'vpc-0864541de209bc08e (my-vpc)'. The 'Subnet' is 'subnet-0556c3a26c5862914'. The 'Auto-assign public IP' is set to 'Enable'. The 'Firewall (security groups)' section has 'Create security group' selected. The 'Security group name' is 'webserver'. The 'Description' is 'webserver'. The 'Summary' panel on the right shows 'Number of instances' as 1, 'Software Image (AMI)' as 'Amazon Linux 2023 AMI', 'Virtual server type (instance type)' as 't2.micro', and 'Firewall (security group)' as 'New security group'. A 'Free tier' notice is also present.

Figure 5.2.12: Network settings

The above figure 5.2.12 shows editing of network settings with my default my VPC and security group names.

## 5.2.13 Inbound rules:

The screenshot shows the 'Inbound Security Group Rules' page in the AWS Management Console. It displays two rules for a security group. Rule 1 is for 'ssh' (Type), 'TCP' (Protocol), and '22' (Port range). The 'Source type' is 'My IP' and the 'Name' is '152.58.236.143/32'. Rule 2 is for 'HTTP' (Type), 'TCP' (Protocol), and '80' (Port range). The 'Source type' is 'Anywhere' and the 'Name' is 'public'. The 'Summary' panel on the right shows 'Number of instances' as 1, 'Software Image (AMI)' as 'Amazon Linux 2023 AMI', 'Virtual server type (instance type)' as 't2.micro', and 'Firewall (security group)' as 'New security group'. A 'Free tier' notice is also present.

Figure 5.2.13: Inbound Rules

The above figure 5.2.13 shows editing of network settings by giving inbound security group rules and required protocol types.

## 5.2.14 Binbash commands:

Metadata response hop limit: 2

Allow tags in metadata: Select

User data - optional

```
#!/bin/bash
sudo su
yum update -y
yum install httpd -y

# Start and enable httpd
systemctl start httpd
systemctl enable httpd

# Move to web root
cd /var/www/html

# Create toy store website HTML file
cat <<EOF>> index.html
<DOCTYPE html>
```

Summary

- Number of instances: 1
- Software Image (AMI): Amazon Linux 2023 AMI
- Virtual server type (instance type): t2.micro
- Firewall (security group): New security group
- Storage (volumes): 1 volume(s) - 8 GiB

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GB of snapshots, and 100 GB of bandwidth to the internet.

Launch instance

Figure 5.2.14: BINBASH commands

The above figure 5.2.14 displays the advance settings and gives bin bash commands to create static website.

## 5.2.15 Binbash commands:

Summary

- Number of instances: 1
- Software Image (AMI): Amazon Linux 2023 AMI
- Virtual server type (instance type): t2.micro
- Firewall (security group): New security group
- Storage (volumes): 1 volume(s) - 8 GiB

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GB of snapshots, and 100 GB of bandwidth to the internet.

Launch instance

Figure 5.2.15: BINBASH commands

The above figure 5.2.15 shows static website code by adding bin bash commands then our website infrastructure is created.

## 5.2.16 Launch instances:

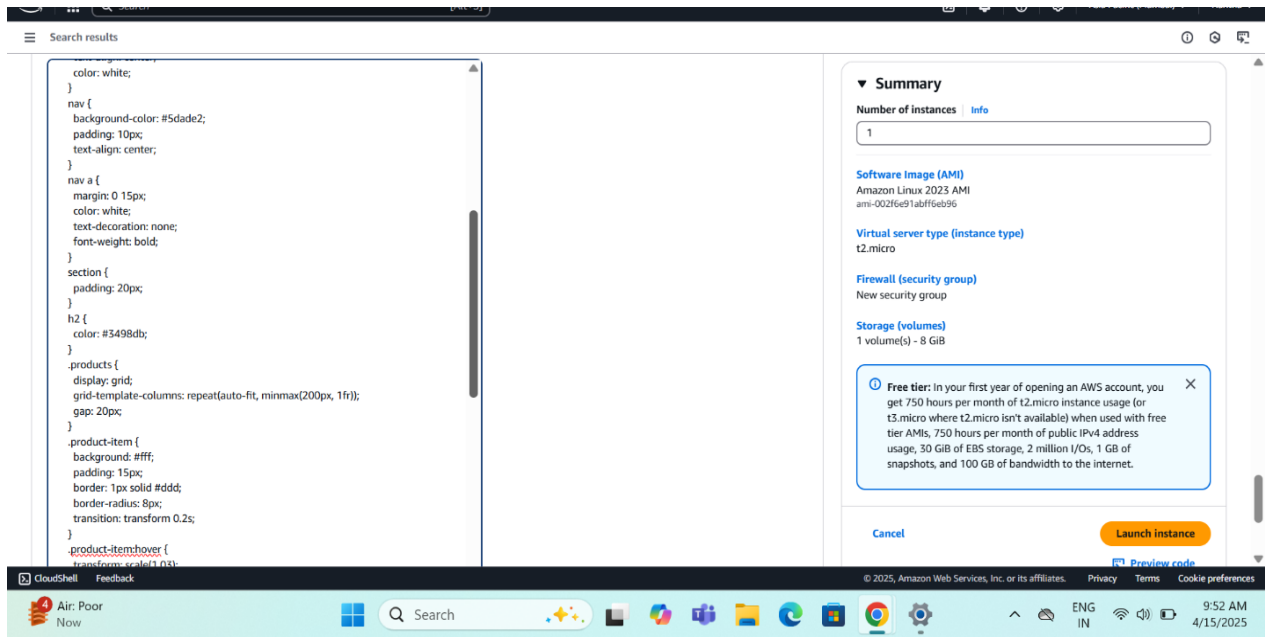


Figure 5.2.16: Launch instances

The above figure 5.2.16 shows the bin bash commands, now click launch instances.

## 5.2.17 Webserver Address:

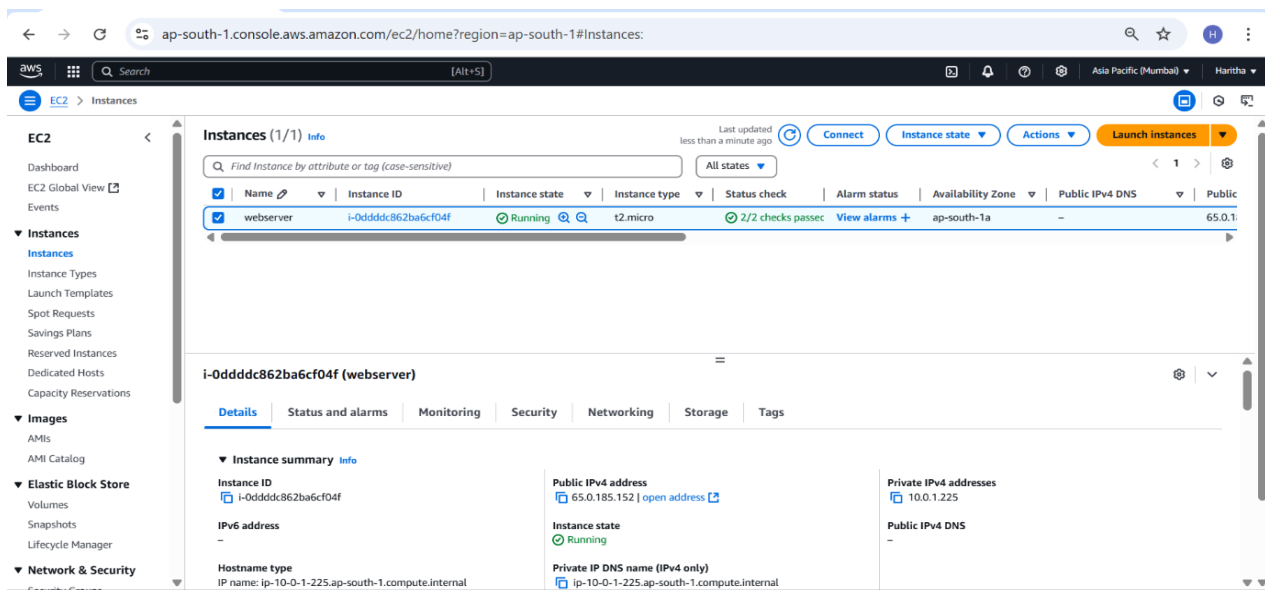


Figure 5.2.17: Webserver address

The above figure 5.2.17 shows Webserver is created tap on it copy the public IPv4 Address of Webserver, paste in chrome browser.

## 5.2.18 Static website:

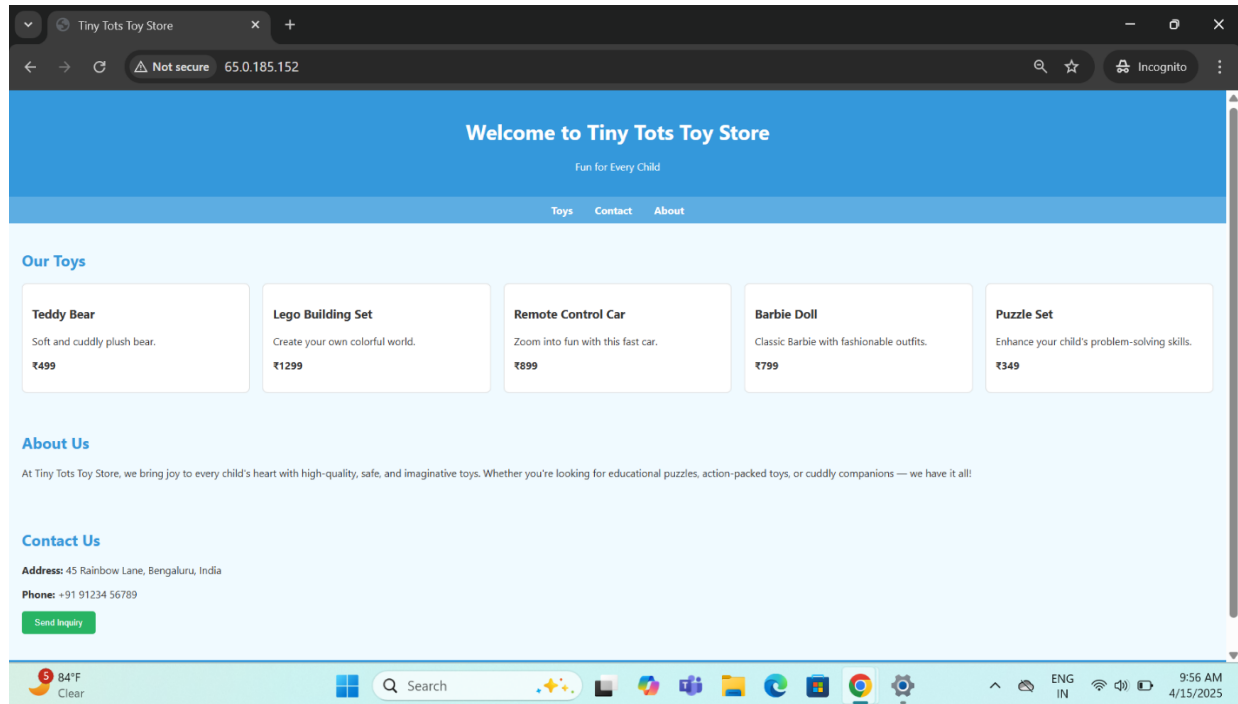


Figure 5.2.18: Static website

The above figure 5.2.18 shows Static website is created successfully by pasting the Public IPv4 address of webserver in browser.

## 5.2.19 Database:

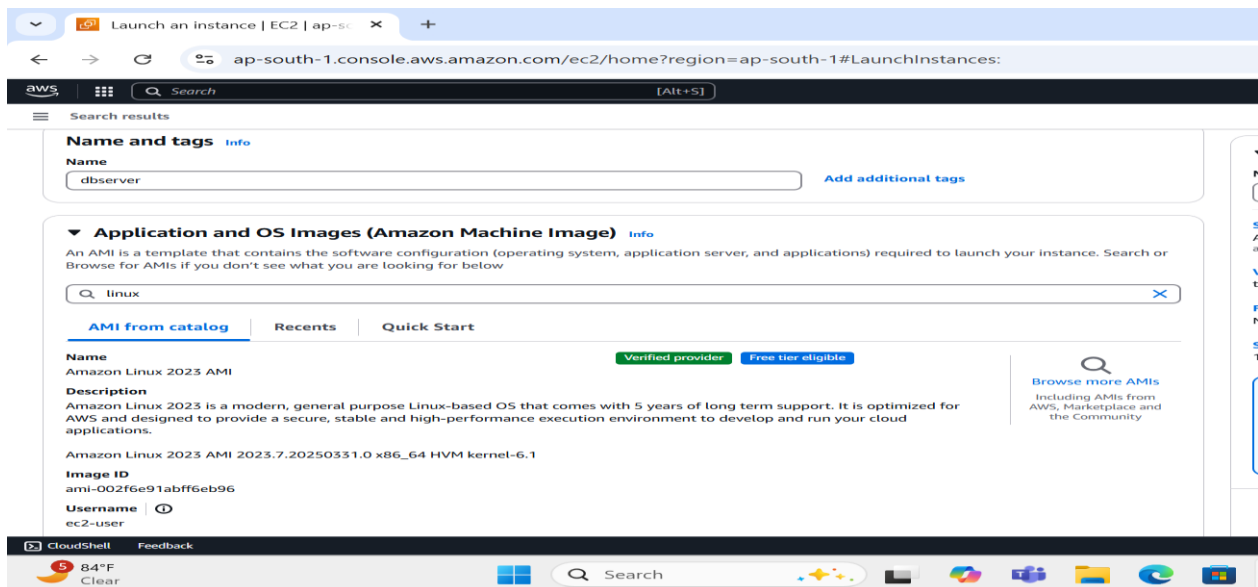


Figure 5.2.19: Database

The above figure displays shows the creation of database by giving tags and Linux OS.

## 5.2.20 Network settings:

ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#LaunchInstances:

Search results

### Network settings

VPC - required

vpc-0864541de209bc08e (my-vpc)

Subnet

subnet-02fe0a0fd1f4d9d8e my-pvtsubnet

Auto-assign public IP

Disable

Firewall (security groups)

Create security group

Select existing security group

Security group name - required

dbserver

Description - required

dbserver

### Summary

Number of instances

1

Software Image (AMI)

Amazon Linux 2023 AMI

Virtual server type (instance type)

t2.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GiB of snapshots, and 100 GiB of bandwidth to the internet.

Figure 5.2.20: Network settings

The above figure 5.2.20 shows the editing of network settings with default VPC, private subnet and security group name.

## 5.2.21 Inbound rules:

Inbound Security Group Rules

Security group rule 1 (TCP, 3306, 10.0.1.0/24, pub-cidr)

Type

MySQL/Aurora

Protocol

TCP

Port range

3306

Source type

Custom

Source

10.0.1.0/24

Description - optional

pub-cidr

Security group rule 2 (TCP, 22, 152.58.236.143/32, admin)

Type

ssh

Protocol

TCP

Port range

22

Source type

My IP

Name

152.58.236.143/32

Description - optional

admin

### Summary

Number of instances

1

Software Image (AMI)

Amazon Linux 2023 AMI

Virtual server type (instance type)

t2.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GiB of snapshots, and 100 GiB of bandwidth to the internet.

Cancel

Launch instance

Figure 5.2.21: Inbound Rules

The above figure 5.2.21 displays the creation of database servers by giving tag, network settings and inbound rules.



## 5.2.22 Database Server:

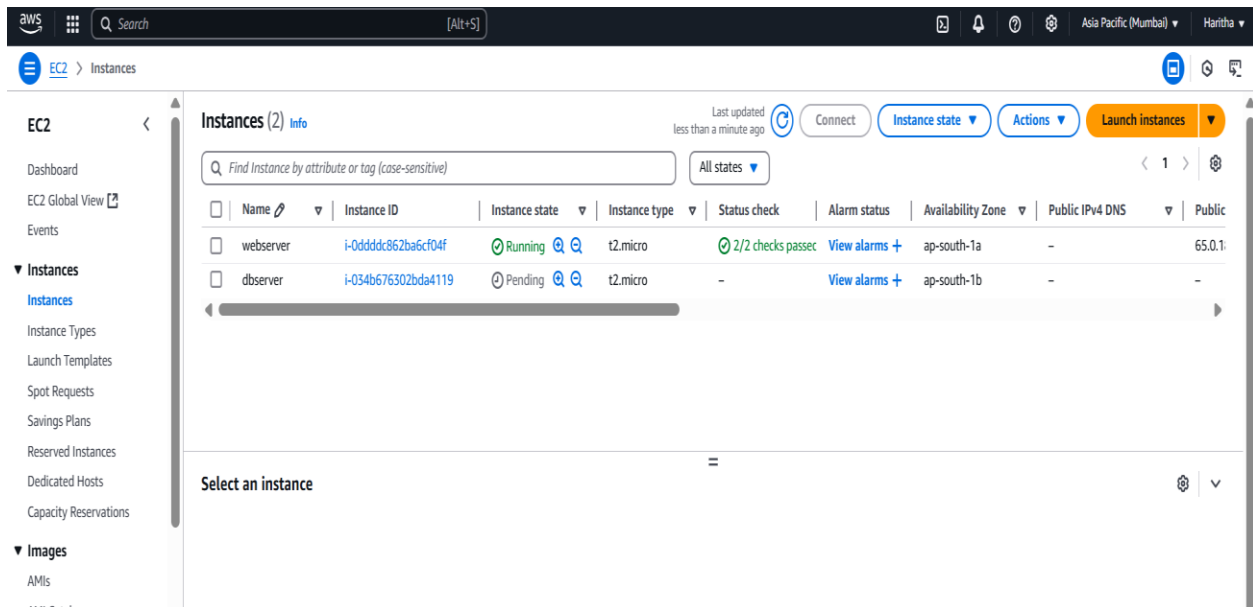


Figure 5.2.22: Database server

The above figure 5.2.22 shows database server is created successfully with instance id, type and other information.

## 5.2.23 Jump box:

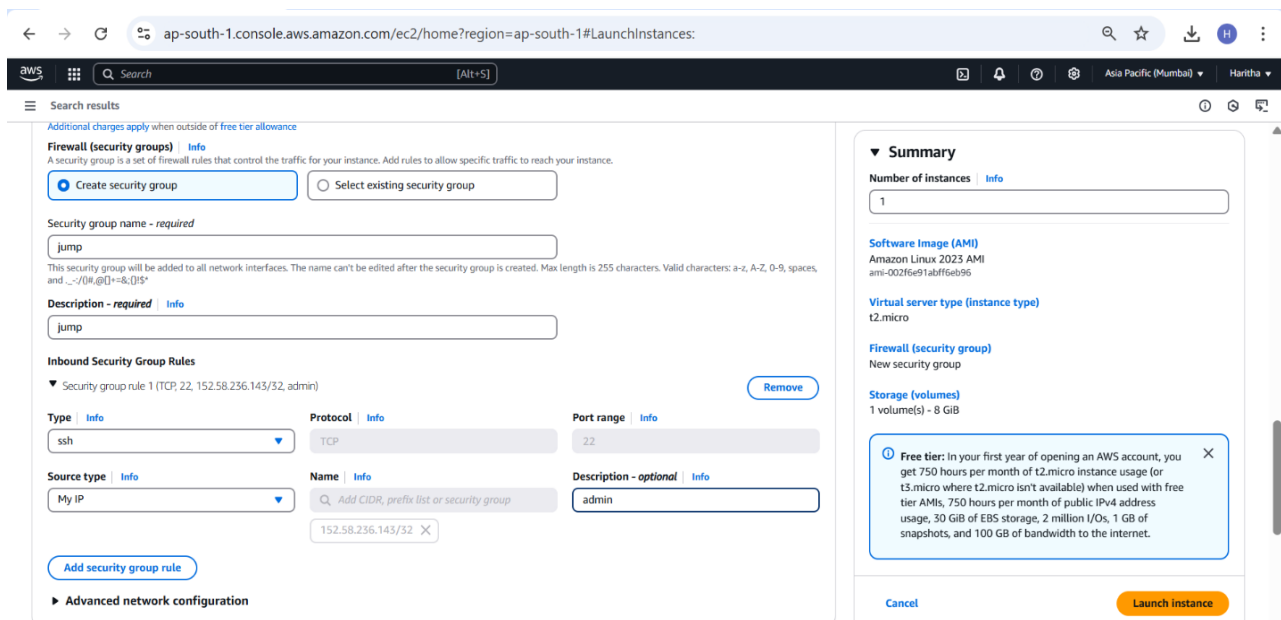


Figure 5.2.23: Jump box

The above figure 5.2.23 shows Jump box is created by giving tag, key pair creation, default my VPC, public subnet, security group name, and inbound rules are SSH and MY IP.

## 5.2.24 Jumpbox:

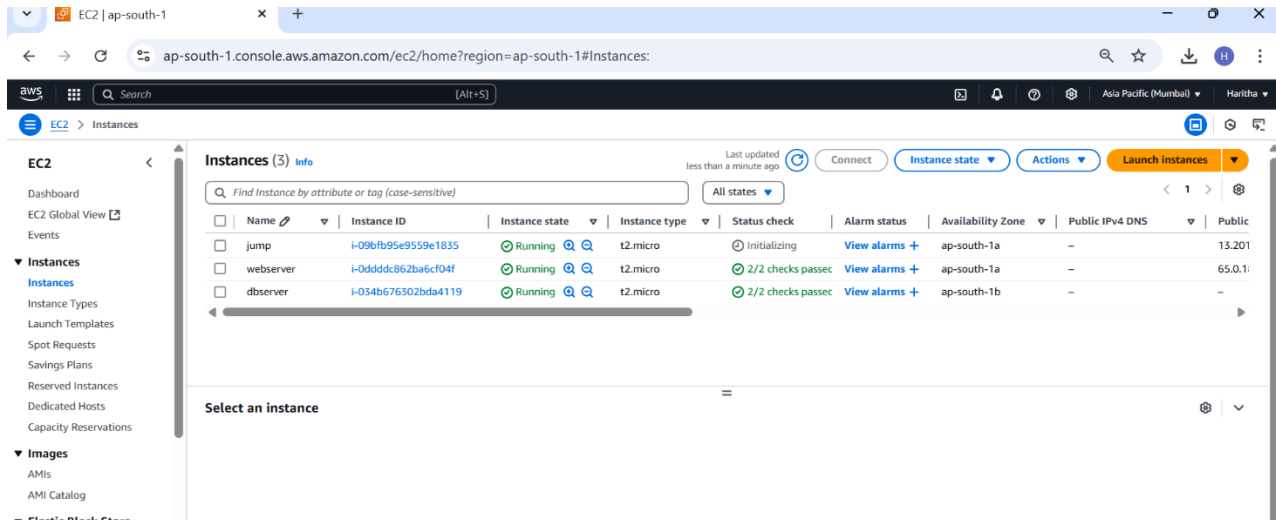


Figure 5.2.24: Jump box

The above figure 5.2.24 shows jump box creation is completed with instance id, type and other information.

## 5.2.25 Jumpbox address:

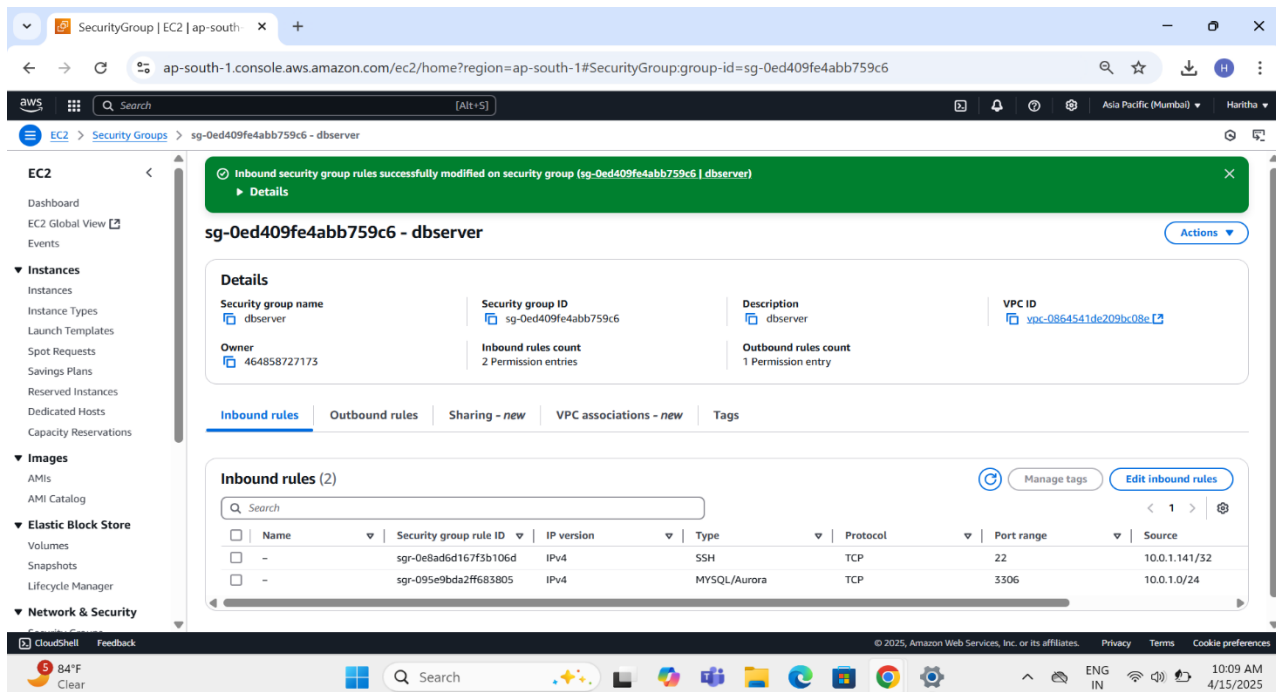


Figure 5.2.25: Jump box address

The above figure 5.2.25 shows private IP address of jump box go to database server, select jump Box go to security groups select edit, Paste the private IP of jump and give description.

## 5.2.26 SSH client:

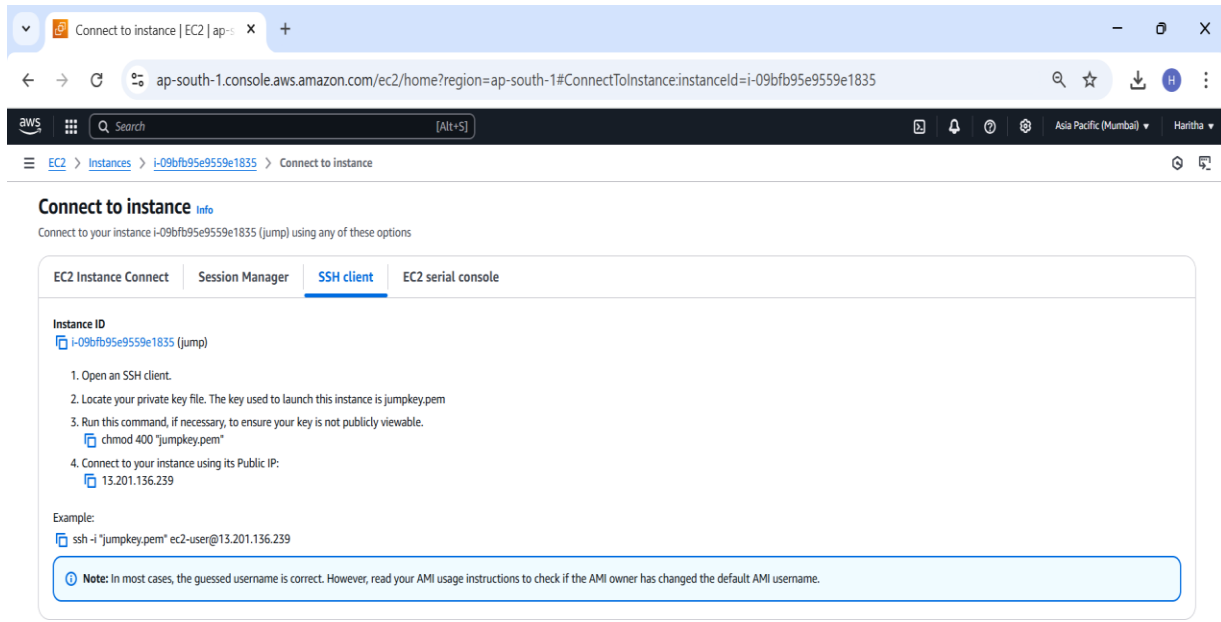


Figure 5.2.26: SSH Client

The above figure 5.2.26 shows jump box in ec2 instance tap on connect go to ssh client copy example Go to puttygen to convert pem file to ppk after that open putty login as ec2-user.

## 5.2.27 putty:



Figure 5.2.27: Putty

The above figure 5.2.27 displays the commands and open WINSCP app and add database to ec2 user. In WinSCP they give hostname as jump box, and username as ec2 user give jump file.

### 5.2.28 putty:

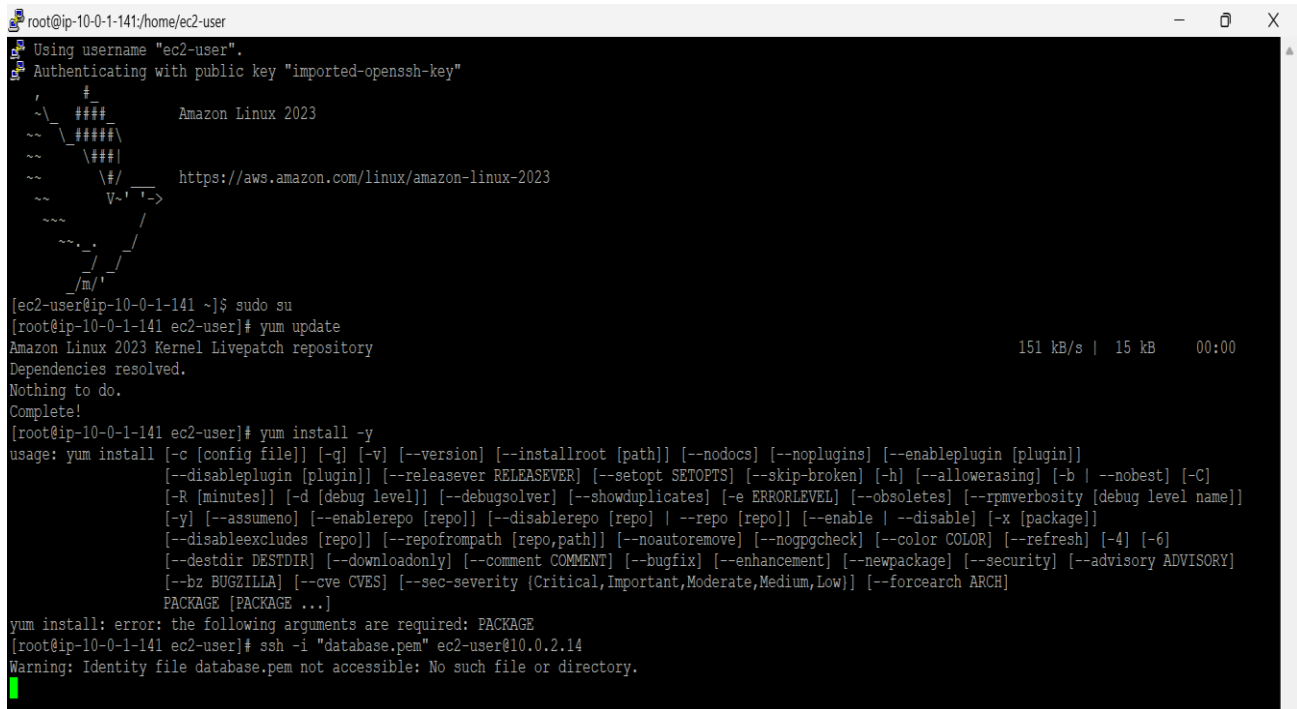


Figure 5.2.28: Putty

The above figure 5.2.28 shows putty you can see public key is authenticating and type commands like yum commands.

### 5.2.29 Putty:

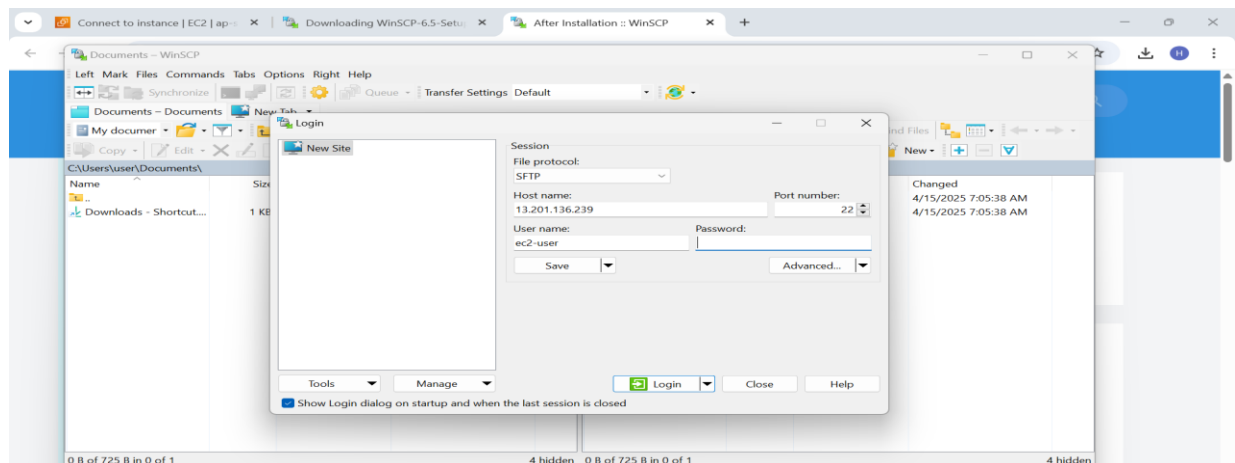


Figure 5.2.29: WinSCP

The above figure 5.2.29 shows that the database example to root server we use the WINSOCP app give public IP as hostname and login as ec2 user.

## 5.2.30 Winscp:

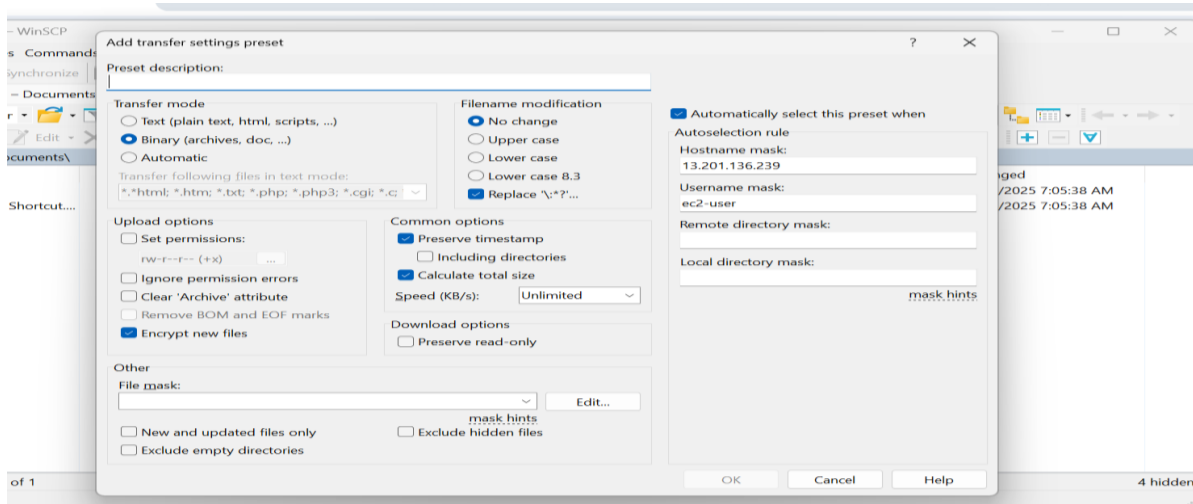


Figure 5.2.30: WINSCP

The above figure 5.2.30 shows WinSCP tap on password you get add transfer settings preset in that give database file and save the changes for successful transfer.

## 5.2.31 Winscp:

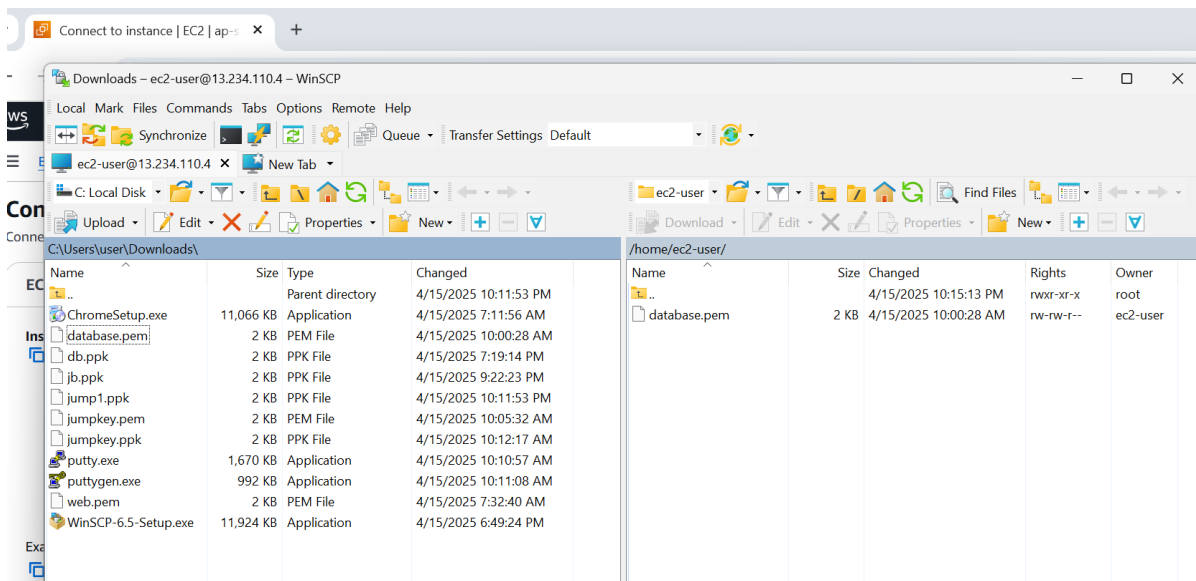


Figure 5.2.31: WinSCP

The above figure 5.2.31 shows that successfully a database file is added to WinSCP now our database file is in root server.

## 5.2.32 NAT GATEWAY:

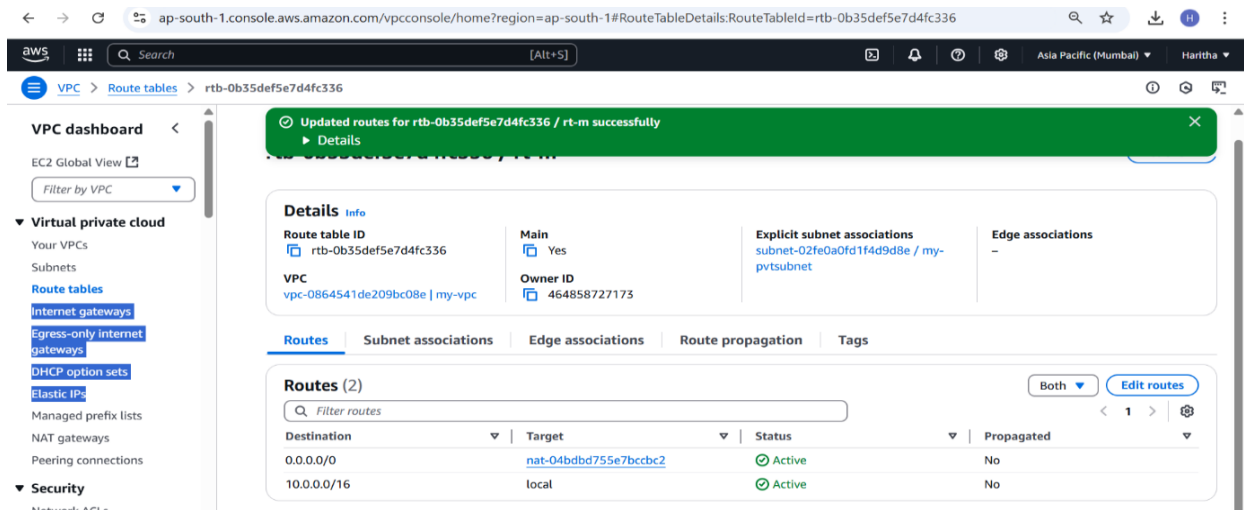


Figure 5.2.32: NAT Gateway

The above figure 5.2.32 shows NAT gateway, it is created by control the overflow of traffic in network. NAT GATE is created by giving name, public subnet, allocate elastic IP address go to route tables route-main select subnet associations edit it select private subnet and save. Go to routes edit it by adding route give destination address and save.

## 5.2.33 Putty:



Figure 5.2.33: Putty

The above figure 5.2.33 shows about Checking internet connection and type command in putty like yum install git -y, then paste the database example in putty now the database example is transfer to root server

## CHAPTER 6

### TESTING

#### 6.1 Testing VPC:

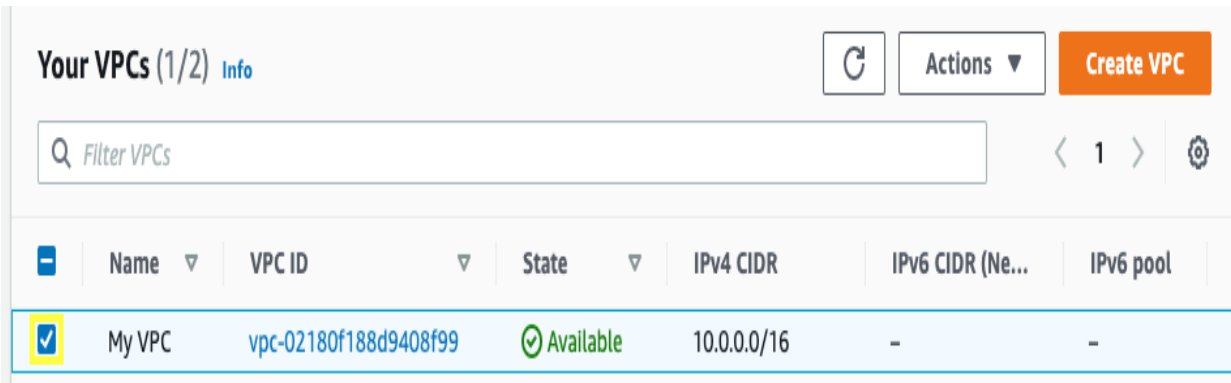


Figure 6.1: Testing VPC

The above figure 6.1 shows my virtual private cloud is successfully created by giving the tag name, IP4 CIDR range. By using this default VPC we can start the process to create the Subnets, routes, internet gate and NAT gateway.

#### 6.2 Testing Subnet:

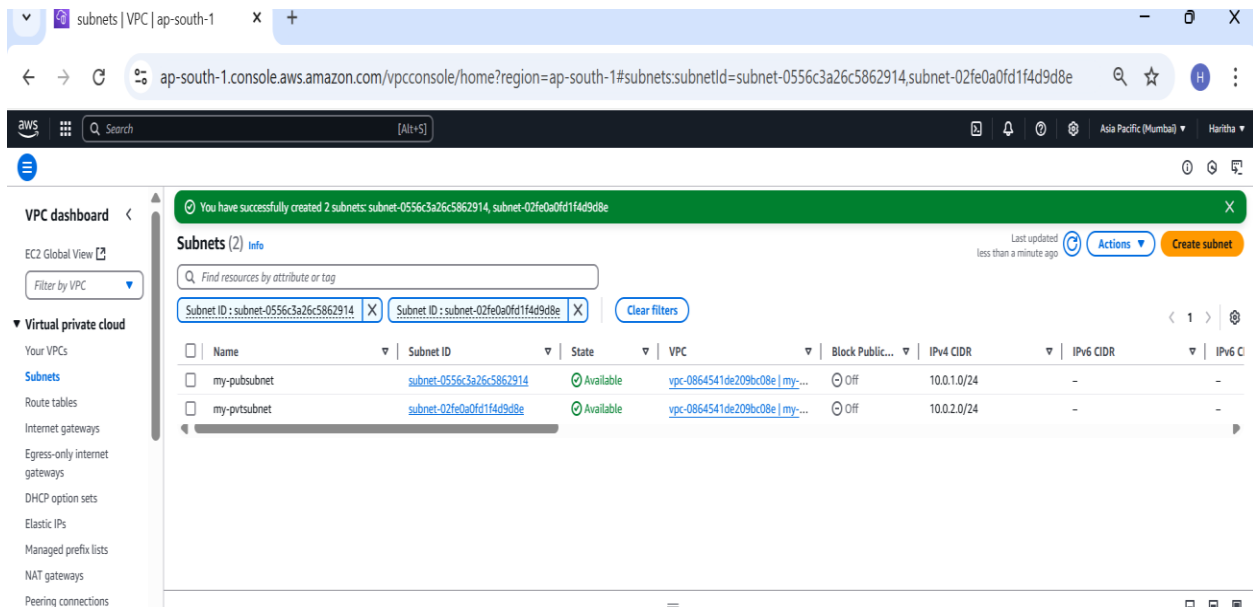


Figure 6.2: Testing Subnet

The above figure 6.2 shows public subnet and private subnets are created by giving default my VPC, availability zones and CIDR ranges.

## 6.3 Testing IGW:

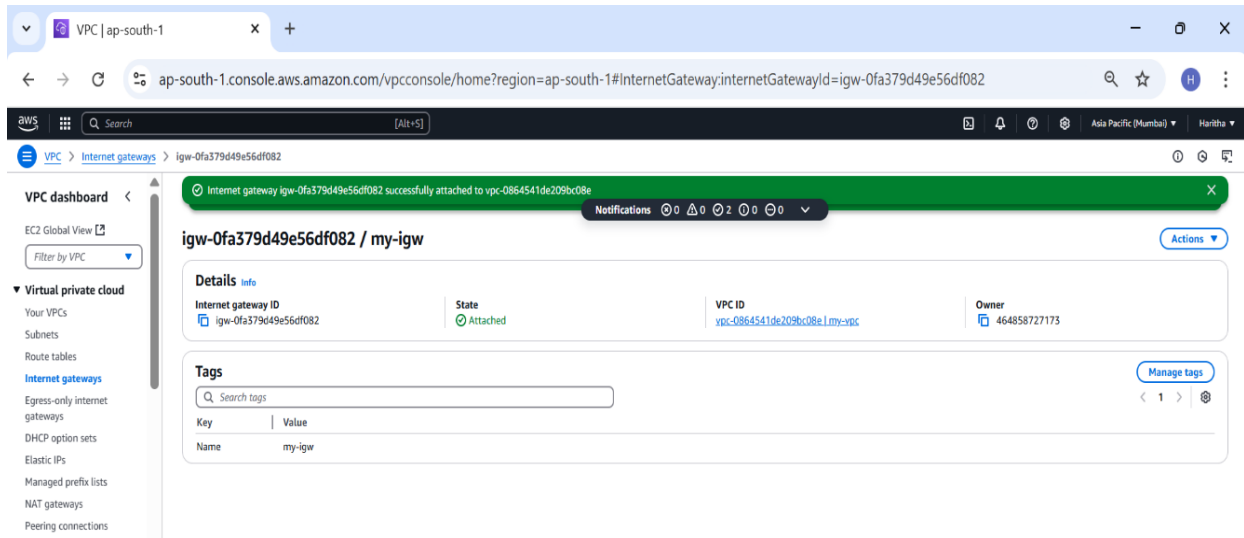


Figure 6.3: Testing IGW

The above figure 6.3 shows Internet gateway is created after successful creation go to Actions tap on attach to a VPC it will attach to my default VPC.

## 6.4 Testing Routes:

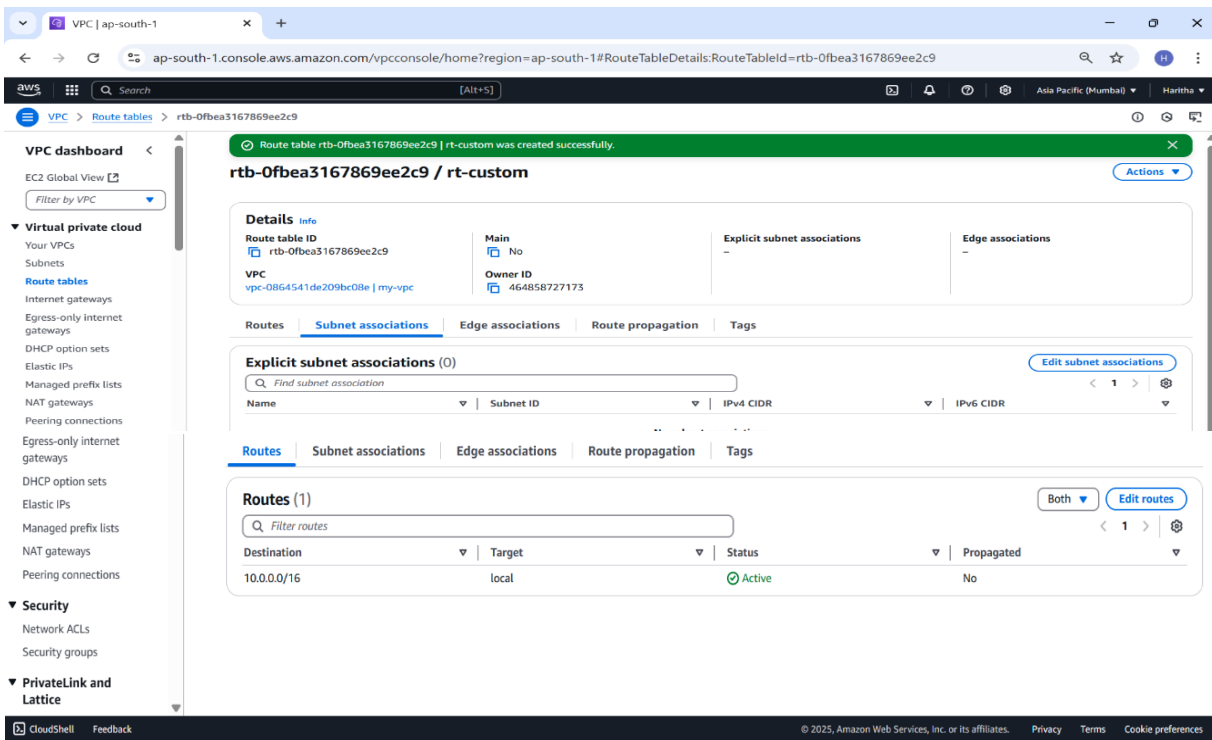


Figure 6.4: Testing Routes

The above figure 6.4 shows route-main, route-custom are created by editing subnet associations and editing routes.



## 6.5 Testing Webserver and Database:

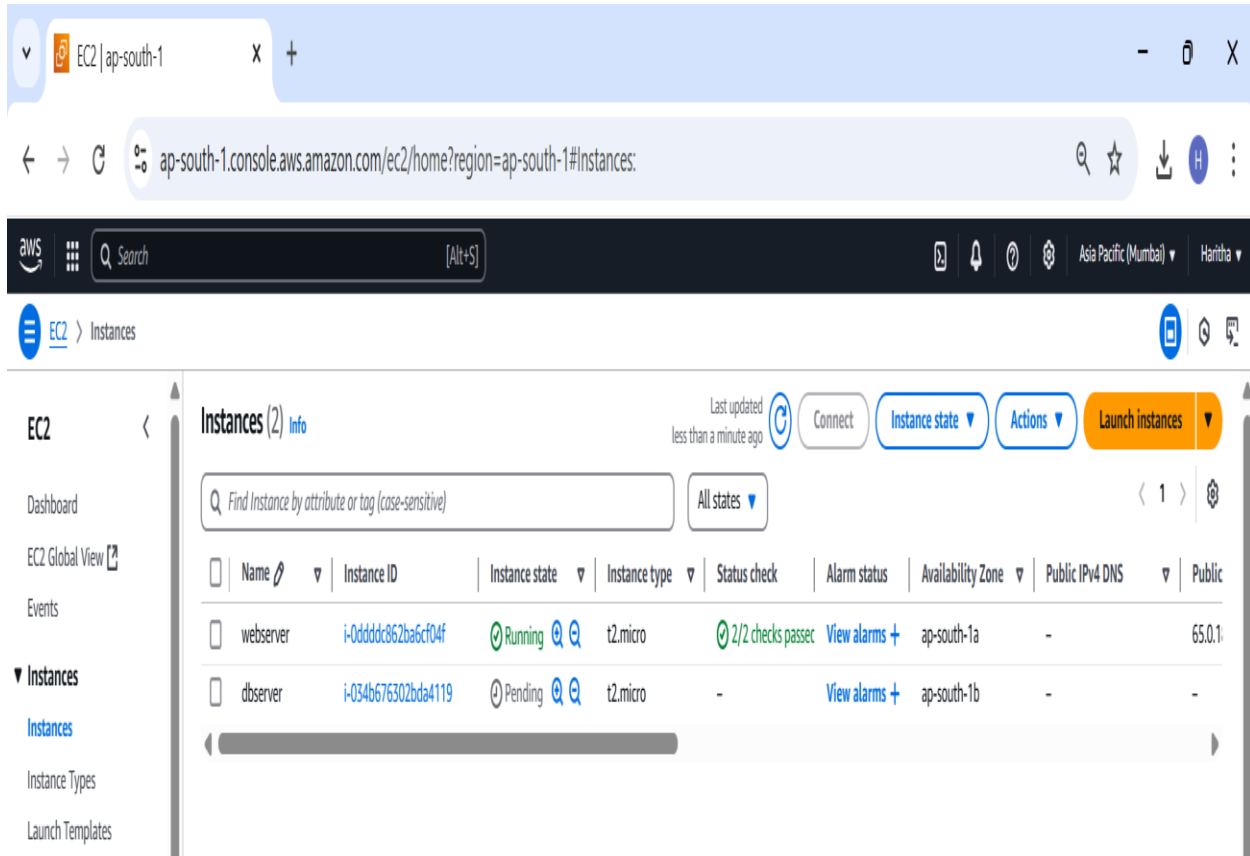


Figure 6.5: Webserver and Database

The above figure 6.5 shows the testing of webserver and database. In an AWS VPC (Virtual Private Cloud) architecture, setting up a testing web server and database involves creating a secure and isolated environment for development or quality assurance purposes. First, a VPC is created with a custom IP range (CIDR block), such as 10.0.0.0/16, to logically separate the testing infrastructure.

Within the VPC, public and private subnets are configured—public subnets are used for resources that need internet access, such as a testing web server, while private subnets are reserved for internal components like databases that should not be exposed to the internet.

The testing web server, typically hosted on an EC2 instance, resides in the public subnet. It can run web services using Apache.

For the database, AWS offers two main options: using Amazon RDS for managed relational databases or deploying a database manually on an EC2 instance.

## 6.6 Testing Jump Box:

Search results

Additional charges apply when outside of free tier allowance

Firewall (security groups) | Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

Create security group

Select existing security group

Security group name - required

jump

This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and \_./@.#[]+=&|!\$\*

Description - required | Info

jump

Inbound Security Group Rules

▼ Security group rule 1 (TCP: 22, 152.58.236.143/32, admin)

Type | Info

ssh

Protocol | Info

TCP

Port range | Info

22

Name | Info

My IP

Add CIDR, prefix list or security group

152.58.236.143/32 ✕

Description - optional | Info

admin

Add security group rule

► Advanced network configuration

Summary

Number of instances | Info

1

Software image (AMI)

Amazon Linux 2023 AMI  
ami-002f6e91abff6eb96

Virtual server type (instance type)

t2.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Free tier:

In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GB of snapshots, and 100 GB of bandwidth to the internet.

Launch instance

Figure 6.6: Jump box

The above figure 6.6 displays the editing of the network settings with public subnet and giving inbound security group rules as type SSH, MY IP. Paste the private IP of jump box in database security groups next connect the jump box to putty

## 6.7 Testing NAT GATEWAY:

Internet gateways  
Egress-only internet gateways  
DHCP option sets  
Elastic IPs  
Managed prefix lists  
NAT gateways  
Peering connections  
▼ Security  
Network ACLs

Routes
Subnet associations
Edge associations
Route propagation
Tags

Routes (2)

Both ▼ Edit routes

< 1 > ⚙

| Destination ▼ | Target ▼                              | Status ▼ | Propagated ▼ |
|---------------|---------------------------------------|----------|--------------|
| 0.0.0.0/0     | <a href="#">nat-04bdbd755e7bccbc2</a> | ✔ Active | No           |
| 10.0.0.0/16   | local                                 | ✔ Active | No           |

Figure 6.7: NAT Gateway

The above figure 6.7 shows NAT GATEWAY ,It is created by giving tag, subnet, elastic IP allocation, route editing saves the changes, type yum commands in putty.

## CHAPTER 7

### RESULTS

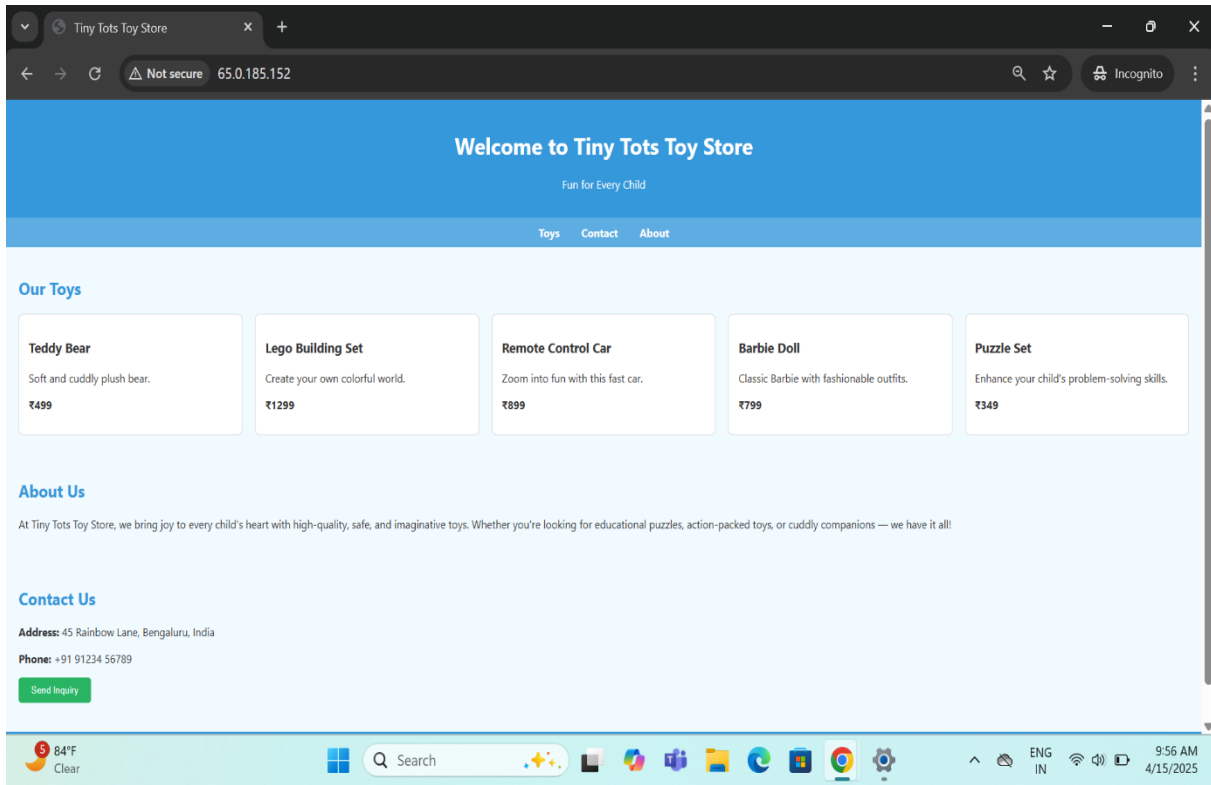


Figure 7: Website

The figure 7 displays the static website. Its primary role is to display a collection of products based on user navigation, search queries, or selected categories. For a toy store, this page must be both visually engaging and functionally intuitive, catering to parents, gift buyers, and even children browsing alongside adults. This page provides users with a quick overview of available toys, including key details such as product name, a brief description, and price. Each product is presented in a card layout to maintain a clean and organized structure.

This layout helps users scan multiple items efficiently, promoting a smoother decision-making process. From a user experience perspective, the results page is designed with simplicity and clarity in mind. Important information is prioritized, and visual clutter is minimized to ensure that the user's attention remains focused on the products. Adding enhancements such as filtering options (by price, category, or age group), sorting mechanisms (like “Price: Low to High” or “Most Popular”), and product images significantly improves usability.

The results page also plays a key role in the conversion funnel. It acts as a bridge between exploration and purchase, enabling users to move from discovery to product detail and eventually to checkout. Features like “Add to Cart” buttons, ratings, and quick view options can encourage faster purchasing decisions.

## CHAPTER 8

### CONCLUSION

AWS Virtual Private Cloud (VPC) is a fundamental component of modern cloud infrastructure, providing a secure, customizable, and scalable environment for deploying AWS resources. It allows organizations to create isolated virtual networks that mimic traditional on-premises environments while benefiting from the flexibility of the cloud.

Within a VPC, resources can be organized into public and private subnets to enhance security and efficiency—public subnets host internet-facing services like web servers, while private subnets are reserved for sensitive components such as databases.

VPC supports various connectivity options, including Internet Gateways for external access, NAT Gateways for enabling internet access from private subnets, VPC Peering and Transit Gateways for connecting multiple VPCs, and VPN or Direct Connect for secure hybrid cloud integration. It also incorporates robust security features, such as Security Groups and Network ACLs for traffic filtering, and integrates with AWS tools like CloudWatch, VPC Flow Logs, and IAM for monitoring and access control.

Furthermore, VPC supports high availability and scalability through features like Auto Scaling, Elastic Load Balancing, and multi-AZ deployments. Overall, AWS VPC empowers organizations to build reliable and secure network architectures in the cloud, making it an essential element in any cloud-based solution.

## CHAPTER 9

### FUTURE SCOPE AND ENHANCEMENTS

As cloud adoption continues to grow, the AWS VPC architecture can be enhanced and scaled to support more advanced and enterprise-grade requirements. One major area of future development is the integration of automation and Infrastructure as Code (IAC) using tools like AWS Cloud Formation, Terraform, or the AWS CDK. This allows for faster, repeatable, and version-controlled deployment of VPC resources, significantly improving operational efficiency.

Another enhancement is the implementation of advanced security measures, such as AWS Network Firewall and AWS Web Application Firewall, to provide deeper packet inspection and better protection against evolving threats. VPC can also benefit from Zero Trust Architecture principles, enforcing strict identity verification and least privilege access across all components.

For organizations with growing or global infrastructure, the VPC architecture can be expanded using Transit Gateway to simplify the management of multiple VPCs across different regions, ensuring secure and scalable interconnectivity. Implementing Private Link and Interface Endpoints can further enhance security by enabling private connectivity to AWS services and third-party applications without exposing data to the public internet.

Additionally, future enhancements can focus on performance optimization, such as using Amazon Global Accelerator for improving latency and availability or deploying Elastic Load Balancers with intelligent routing. Integrating with machine learning and AI-based monitoring tools for anomaly detection and traffic prediction can further strengthen the architecture.

## **BIBLIOGRAPHY**

- [1]Amazon VPC Lattice Reference Architectures.  
Authors: Pablo Sánchez Carmona, Adam Palmer  
Published: October 16, 2023  
Description: This document provides architecture diagrams and use cases for Amazon vpc.
  
- [2]Centralize Network Connectivity Using AWS Internet Gateway.  
Authors: Mydhili Palagummi, Nikhil Marrapu  
Published: November 06,2015  
Description: This guide describes the simplest configuration in which AWS internet Gateway.
  
- [3]Architectural Patterns to Build End-to-End Data Driven Applications on AWS  
Authors: AWS Architecture Team  
Published: August 3, 2022  
Description: This whitepaper presents architectural patterns for building VPC.